

EXPLAINABLE AI-BASED LUNG DISEASE DETECTION USING SEGMENTATION AND ENSEMBLE DEEP LEARNING

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING (AI & ML)



By

Batch – A15

A. Mounika (322103282004)

G. Aparna (322103282032)

K. Yogitha (322103282047)

M. Veda Vaishnavi(322103282059)

Under the esteemed guidance of

Dr M BHANU SRIDHAR

Professor & HOD

Dept of IT

Department of Computer Science and Engineering (AI&ML)

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

[Approved by AICTE NEW DELHI, Affiliated to ANDHRA UNIVERSITY Vishakhapatnam]

[Accredited by National Board of Accreditation (NBA) for B.Tech CSE, ECE & IT – Valid from 2019-22 and 2022-25]

[Accredited by National Assessment and Accreditation Council (NAAC)– Valid from 2022-27]

Kommadi, Madhurawada, Visakhapatnam–530048

2022–2026

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI &ML)



CERTIFICATE

This is to certify that the project report titled **“EXPLAINABLE AI-BASED LUNG DISEASE DETECTION USING SEGMENTATION AND ENSEMBLE DEEP LEARNING”** is a Bonafide work of following IV B.Tech. II Sem. students in the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Gayatri Vidya Parishad College of Engineering for Women affiliated to ANDHRA UNIVERSITY Vishakapatnam during the academic year 2025-26, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology of this university.

A. Mounika (322103282004)

G. Aparna (322103282032)

K. Yogitha (322103282047)

M. Veda Vaishnavi(322103282059)

Signature of the Guide

Dr M. BHANU SRIDHAR

Professor & HOD

Dept. of IT

Signature of the HOD

Dr. Dwiti Krishna Bebartta

Professor & HOD

Dept. of CSE(AI&ML)

External Examiner

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people who made it possible and whose constant guidance and encouragement crown all the efforts with success.

We feel elated to extend our sincere gratitude to **Dr. M. Bhanu Sridhar**, Professor, for his encouragement all the way during the analysis of the project. His annotations, insinuations, and criticisms are the key to the successful completion of the thesis and to providing us with all the required facilities.

We would like to take this opportunity to extend our gratitude to Project Coordinator, **Dr. K. Purushotam Naidu**, Associate Professor of Computer Science and Engineering - Artificial Intelligence & Machine Learning for making us follow a defined path and for advising us to get better improvements in the project.

We express our deep sense of gratitude and thanks to **Dr. Dwiti Krishna Bebarta**, Professor and Head of the Department of Computer Science and Engineering (AI&ML) for his guidance and expressing his valuable and grateful opinions on the project for its development and for providing lab sessions and extra hours to complete the project.

We would like to take this opportunity to express our profound sense of gratitude to **Dr. R. K. Goswami**, Principal, and **Dr. G. Sudheer**, Vice Principal for allowing us to utilize the college resources thereby facilitating the successful completion of our thesis.

We are also thankful to both the teaching and non-teaching faculty of the Department of Computer Science and Engineering (AI&ML) for giving valuable suggestions for our project.

A. Mounika (322103282004)

G. Aparna (322103282032)

K. Yogitha (322103282047)

M. Veda Vaishnavi(322103282059)

TABLE OF CONTENTS	
TOPICS	PAGE NO
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	iii
LIST OF SCREENS	iv
LIST OF ACRONYMS	v
1. INTRODUCTION	1-3
1.1 MOTIVATION	1
1.2 PROBLEM DEFINITION	2
1.3 OBJECTIVE OF PROJECT	2
1.4 LIMITATIONS OF PROJECT	3
1.5 ORGANIZATION OF THESIS	3
2. LITERATURE SURVEY	4-15
2.1 INTRODUCTION	4-9
2.2 EXISTING SYSTEM	10-11
2.3 DISADVANTAGES OF EXISTING SYSTEM	11-12
2.4 PROPOSED SYSTEM	13-15
2.5 CONCLUSION	15
3. REQUIREMENT ANALYSIS	16-25
3.1 INTRODUCTION	16
3.2 SOFTWARE REQUIREMENT SPECIFICATION	16
3.2.1 User Requirement	16
3.2.2 Software Requirement	16-17
3.2.3 Hardware Requirement	19
3.2.4 Non-Functional Requirements	19
3.3 CONTENT DIAGRAM OF PROJECT	20
3.4 ALGORITHMS AND FLOWCHARTS	21-25
3.5 CONCLUSION	25

TABLE OF CONTENTS	
TOPICS	PAGE NO
4. DESIGN	26-37
4.1 INTRODUCTION	26
4.2 UML DIAGRAM	26,27
4.2.1 Use Case Diagram	27,28
4.2.2 Class Diagram	29-31
4.2.3 Sequence Diagram	32-34
4.3 MODULE DESIGN AND ORGANIZATION	35-37
4.4 CONCLUSION	37
5. IMPLEMENTATION AND RESULTS	38-65
5.1 INTRODUCTION	38-40
5.2 EXPLAINATION OF KEY FUNCTIONS	41
5.2.1 Deep Learning Models Used	41-44
5.2.2 Explanation Of The Data Used	45-47
5.2.3 Training The Model	48
5.3 METHOD OF IMPLEMENTATION	49
5.3.1 Sample Code	49-55
5.3.2 Results And Analysis	55-59
5.3.3 Output Screens	60-64
5.4 CONCLUSION	65
6. TESTING AND VALIDATION	66-75
6.1 INTRODUCTION	66-72
6.2 DESIGN OF TEST CASES AND SCENARIOS	73,74
6.3 VALIDATION TESTING	74,75
6.4 CONCLUSION	76
7. CONCLUSION	77
8. REFERNCES	78

ABSTRACT

Chest X-ray imaging is widely used for the early detection and diagnosis of lung diseases; however, manual interpretation can be time-consuming and subject to variability among radiologists. This project presents an explainable deep learning framework for automated multi-label lung disease detection from chest X-ray images. The proposed system employs two convolutional neural network architectures, EfficientNet-B0 and ResNet-101, trained on the NIH Chest X-ray dataset to classify fourteen thoracic diseases. To address class imbalance, a weighted Binary Cross-Entropy loss function is utilized during training. In addition, a U-Net model trained on the JSRT dataset is used for lung region segmentation to provide anatomical context and visualization. The predictions from the classification models are combined using an ensemble strategy to improve robustness and performance. To enhance interpretability, Grad-CAM is applied to generate visual explanations highlighting the regions influencing the model's predictions. Furthermore, a Retrieval-Augmented Generation (RAG) based chatbot is integrated to provide disease-related information using a FAISS vector database and an Ollama language model. The proposed system offers an explainable and user-friendly approach for assisting medical professionals in lung disease diagnosis. The Best model selection was done based on accuracy, mean accuracy, f1 score. The ensemble model performed well with 83% accuracy and 76% mean Auc and 23% f1 score.

Keywords: Chest X-ray Analysis, Lung Disease Detection, U-Net Segmentation, EfficientNet, ResNet-101, Grad-CAM, RAG Chatbot.

LIST OF FIGURES

S.No.	Figure No.	Figure Name	Page No
1	Fig 2.1	Hybrid Model Architecture	14
2	Fig 3.1	System Architecture	20
3	Fig 3.2	Flowchart of model training & prediction	23
4	Fig 3.3	Flowchart for RAG based chatbot	24
5	Fig 4.1	Relationship Diagram	27
6	Fig 4.2	Use Case Diagram	28
7	Fig 4.3.1	Class Diagram of chest x-ray diagnosis	30
8	Fig 4.3.2	Class Diagram of RAG chatbot	31
9	Fig 4.4	Sequence Diagram	33
10	Fig 4.5.1	System Workflow of chest x-ray diagnosis	36
11	Fig 4.5.2	System Workflow of RAG chatbot	37
12	Fig 5.1	Structure of CNN	41
13	Fig 5.2	The structure of RESNET-101	43
14	Fig 5.3	The Structure of EfficientNet-b0	43
15	Fig 5.4	Sample Dataset of NIH chest x-ray	45
16	Fig 5.5	Sample Dataset of JSRT	46
17	Fig 5.6	Training and validation curves Resnet-101	55
18	Fig 5.7	Training and validation curves Efficient Net b0	56

LIST OF TABLES

S.No.	Table No.	Table Name	Page No
1	Table 1	Table for Literature Survey	7-9
2	Table 2	Packages Used	18
3	Table 3	Hardware Requirements	19
4	Table 4	Mean AUC	56
5	Table 5	Accuracy and F1-Score	56
6	Table 6	Accuracy of each disease	57
7	Table 7	Types of Tests	72
8	Table 8	Test Cases	73

LIST OF SCREENS

S.No.	Screen No.	Screen Name	Page No
1	Screen 1	Kaggle Interface	38
2	Screen 2	Colab Interface	39
3	Screen 3	Visual Studio Code Interface	40
4	Screen 4	Home page	60
5	Screen 5	Pipeline of chest x-ray analysis	60
6	Screen 6	Signup page	61
7	Screen 7	Login page	61
9	Screen 8	RAG chatbot dashboard	62
10	Screen 9	X-ray analysis dashboard	62
10	Screen 10	Result page	63
11	Screen 11	Visualization	63
12	Screen 12	Rag chatbot prediction	64
13	Screen 13	About the Model	64

LIST OF ACRONYMS

ABBREVIATIONS	ACRONYMS
CNN	Convolutional Neural Network
U-Net	Convolutional Network for Image Segmentation
Grad-CAM	Gradient-weighted Class Activation Mapping
AUC	Area Under the Curve
F1 Score	Harmonic mean of Precision and Recall
API	Application Programming Interface
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
Embedding	Vector representation of text/data
FAISS	Facebook AI Similarity Search
ResNet-101	Residual Network with 101 layers
EfficientNet-B0	Efficient Convolutional Neural Network
Grad-CAM	Gradient-weighted Class Activation Mapping

1. INTRODUCTION

Lung diseases are among the major health concerns worldwide, and early detection plays a crucial role in effective treatment and patient care. Chest X-ray imaging is one of the most commonly used diagnostic techniques due to its affordability and availability. However, manual interpretation of X-ray images is time-consuming and depends heavily on the expertise of radiologists, which may lead to delays and inconsistencies in diagnosis.

With the advancement of deep learning, automated systems have shown significant potential in medical image analysis. In this project, an **explainable lung disease detection system** is proposed using deep learning models such as ResNet-101 and EfficientNet-B0 for multi-label classification. A U-Net model is used for lung segmentation to focus on relevant regions, and Grad-CAM is applied to provide visual explanations of predictions. Additionally, a **RAG-based chatbot** is integrated to deliver context-aware medical information and improve user interaction. The system is designed to be accurate, interpretable, and user-friendly, supporting both diagnosis and understanding.

1.1 MOTIVATION

Lung diseases represent a significant global health challenge, and timely diagnosis is essential for effective treatment and improved patient outcomes. Chest X-ray imaging is widely used for detecting various thoracic conditions; however, the process of manual interpretation is often time-consuming and requires expert radiologists. In many situations, especially in resource-constrained environments, the lack of skilled professionals can lead to delayed diagnosis and increased risk to patients.

At the same time, advancements in deep learning and computer vision have demonstrated strong potential in automating medical image analysis. Despite this progress, many existing systems lack interpretability and function as black boxes, making it difficult for medical professionals to trust their predictions. Additionally, patients often do not have access to clear and reliable information regarding their diagnosis.

These challenges motivated the development of an **explainable and intelligent system** that combines deep learning-based lung disease detection with visual explanations and an interactive RAG-based chatbot. The aim is to create a solution that not only improves diagnostic efficiency but also enhances transparency, user understanding, and accessibility in healthcare.

1.2 PROBLEM DEFINITION

Accurate detection of lung diseases from chest X-ray images is a complex task due to the subtle variations between different thoracic conditions and the dependency on expert radiologists for interpretation. Traditional diagnostic methods are time-consuming and difficult to scale, particularly in resource-limited settings. Although deep learning models such as convolutional neural networks have shown promising results in classification, many existing systems lack interpretability and fail to provide clear insights into their predictions. Additionally, most approaches do not incorporate anatomical context or offer user interaction, which limits their practical usability. Integrating advanced techniques like CNN-based classification, lung segmentation, and explainability methods improves performance but introduces challenges such as increased computational complexity and the need for efficient optimization. Therefore, there is a need to develop a robust, explainable, and scalable system that can accurately detect lung diseases from chest X-ray images while also providing meaningful visual explanations and interactive support through a RAG-based chatbot.

1.3 OBJECTIVE OF THE PROJECT

- To develop an automated and efficient system for detecting multiple lung diseases from chest X-ray images using deep learning models such as **ResNet-101 and EfficientNet-B0**. To preprocess and normalize data effectively while preserving temporal features.
- To enhance the interpretability of the model by generating visual explanations using techniques like **Grad-CAM**, enabling better understanding of the prediction results.
- To improve anatomical accuracy by incorporating a **U-Net-based lung segmentation model** to generate visual overlays highlighting lung regions
- To refine prediction results using appropriate **thresholding techniques** in order to reduce false positives and improve overall reliability.
- To integrate a **Retrieval-Augmented Generation (RAG)-based chatbot** that provides detailed, context-aware medical information and supports interactive user queries.
- To design a **scalable and modular system architecture** that ensures efficient processing, easy integration of components, and improved user experience.

1.4 LIMITATIONS OF PROJECT

- The performance of the system is highly dependent on the availability of large and well-labelled chest X-ray datasets; limited or imbalanced data can affect prediction accuracy.
- The integration of multiple deep learning models such as **ResNet, EfficientNet, and U-Net**, along with explainability techniques, increases computational complexity and requires high processing power.
- Model performance may vary based on hyperparameter tuning, and improper configuration can lead to overfitting or reduced generalization.
- The system focuses only on chest X-ray images and may not generalize well to other medical imaging modalities without further training.
- Although Grad-CAM provides visual explanations, the interpretability may still not be fully sufficient for clinical decision-making without expert validation.
- The RAG-based chatbot relies on the quality and coverage of the medical knowledge base; incomplete or outdated data may affect the accuracy of responses.
- The system is designed as an assistive tool and cannot replace professional medical diagnosis or expert consultation.

1.5 ORGANIZATION OF THESIS

A concise overview of the remaining sections of this documentation is provided below:

- In **Chapter 1**, Introduction i.e., describes the motivation, problem definition, objectives, and limitations of the project.
- In **Chapter 2**, Literature Survey i.e., provides an overview of related work, existing models, and approaches used in previous studies.
- In **Chapter 3**, Analysis i.e., deals with the system requirements, functional, and non-functional specifications.
- In **Chapter 4**, it contains the Design of the proposed system.
- In **Chapter 5**, Implementation shows step-by-step process, algorithms, and screenshots of the output.
- In **Chapter 6**, it explains Testing and Validation of the project.
- In **Chapter 7**, it describes the Conclusion of the project.
- In **Chapter 8**, Reference papers, books, and resources consulted during the project are mentioned here.

2. LITERATURE SURVEY

2.1 INTRODUCTION

Chest X-ray imaging plays a vital role in the early detection and diagnosis of lung diseases due to its low cost, wide availability, and fast processing time. However, manual interpretation of chest X-ray images can be time-consuming and may lead to inconsistencies depending on the experience of radiologists. With the rapid advancement of deep learning techniques, automated systems have been developed to assist in medical image analysis and improve diagnostic accuracy. In recent years, several studies have explored the use of convolutional neural networks (CNNs) for lung disease classification from chest X-ray images.

For instance, Ifty et al. (2024) proposed a deep learning-based classification system capable of identifying multiple lung diseases such as pneumonia, COVID-19, and tuberculosis using explainable AI techniques like Grad-CAM and LIME. Similarly, Sarker et al. (2025) introduced a hybrid deep learning framework that combines multiple CNN architectures and datasets to improve classification performance and generalization. Veeramani et al. (2025) further enhanced this approach by integrating lung segmentation with transfer learning models, allowing the system to focus on relevant lung regions and improve interpretability. In addition, Mahamud et al. (2024) developed a DenseNet-based model that uses explainable AI to improve transparency in detecting diseases such as COVID-19 and pneumonia.

Moreover, Naz et al. (2023) presented a CNN-based explainable framework using ResNet models for pulmonary disease detection, highlighting the importance of interpretability in medical diagnosis. A comprehensive review by recent researchers (2024) also emphasized the effectiveness of deep learning methods and the growing importance of explainable AI in medical imaging applications. Although these approaches demonstrate promising results, many existing systems still face challenges such as high computational complexity, lack of segmentation in some models, and limited generalization across different datasets.

Therefore, there is a need for a reliable and explainable system that can effectively detect multiple lung diseases while providing clear visual interpretations and supporting medical understandings.

In [1] *“An Explainable Deep Learning–Based Multi-Label Image Classification for Chest X-Rays”* by Thapa and Kaur (2025), the authors propose a deep learning framework using DenseNet-121 for detecting multiple lung diseases from chest X-ray images. The model performs multi-label classification and incorporates explainable AI techniques such as Grad-CAM, LIME, and DeepLIFT to highlight important regions influencing predictions. The results show improved interpretability and an AUC of approximately 0.87. However, the model suffers from class imbalance and lacks lung segmentation for better anatomical focus.

In [2] *“Comprehensive Lung Disease Detection Using Deep Learning Models and Hybrid Chest X-Ray Data with Explainable AI”* by Sarker et al. (2024), a hybrid deep learning approach is proposed using multiple CNN architectures such as VGG16, DenseNet121, and ResNet50V2. The system combines datasets to improve generalization and incorporates explainable AI techniques for visualization. The results demonstrate high accuracy in detecting multiple lung diseases. However, the model increases computational complexity and does not include segmentation.

In [3] *“Revolutionizing Lung Disease Diagnosis: A Hybrid Deep Learning Framework for Explainable Chest X-Ray Analysis”* by Priyatharsini et al. (2024), the authors introduce a hybrid framework that integrates deep learning with explainable AI to improve diagnostic transparency. The system provides disease predictions along with visual explanations, helping medical professionals understand model decisions. However, the architecture is complex and lacks proper handling of dataset imbalance and segmentation.

In [4] *“CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays”* by Rajpurkar et al. (2017), a DenseNet-121 based deep learning model is proposed for pneumonia detection. The model achieves performance comparable to radiologists, demonstrating the potential of deep learning in medical imaging. However, it focuses only on pneumonia, lacks multi-label classification, and does not provide explainability.

In [5] *“Explainable Lung Disease Classification from Chest X-Ray Images”* by Ifty et al. (2024), the authors present a CNN-based model combined with explainable AI techniques such as Grad-CAM and LIME. The system can detect multiple diseases including COVID-19 and pneumonia and provides visual explanations. However, it does not include segmentation and requires further validation on larger datasets.

In [6] “*Explainable AI-Based Lung Disease Diagnosis Using Chest X-ray Images*” by Veeramani et al. (2025), a deep learning system using transfer learning models such as VGG16, VGG19, and ResNet50 is proposed. The model integrates lung segmentation and explainable AI techniques to improve accuracy and interpretability. However, the system requires high computational resources and further clinical validation.

In [7] “*Explainable AI Model for COVID-19, Pneumonia and Tuberculosis Detection*” by Mahamud et al. (2024), the authors propose a DenseNet-based transfer learning model combined with explainable AI techniques. The system achieves high accuracy and provides better interpretability. However, it focuses on a limited number of diseases and lacks segmentation.

In [8] “*CNN-Based Explainable AI Framework for Pulmonary Disease Detection*” by Naz et al. (2023), a CNN and ResNet-based model is used for detecting pulmonary diseases from chest X-rays. The system incorporates explainable AI techniques to highlight important regions. The results show improved accuracy and interpretability, but the model does not address dataset imbalance and lacks segmentation.

In [9] “*Deep Learning for Pneumonia Detection in Chest X-ray Images: A Review*” (2024), the authors provide a comprehensive overview of various deep learning models used for pneumonia detection. The study highlights the effectiveness of CNN-based models and the importance of explainable AI. However, it is a review paper and does not propose a new model

In [10] “*Precise Zero-Shot Dense Retrieval without Relevance Labels*” (2023), the authors propose a novel approach for dense retrieval that eliminates the need for labelled relevance data. The study demonstrates how zero-shot techniques can effectively improve retrieval performance by leveraging pre-trained language models, making it highly suitable for information retrieval tasks in low-resource settings. However, the work primarily focuses on retrieval optimization and does not directly address end-to-end task-specific applications such as domain-specific prediction or classification systems.

Table 1: Table for Literature Survey

S.No.	Title and Author	Published Journal	Dataset Used	Approach	Outcome
1.	An Explainable Deep Learning–Based Multi-Label Image Classification for Chest X-Rays – Thapa and Kaur (2025)	Procedia Computer Science	CheXpert chest X-ray dataset	DenseNet-121 based deep learning model with Grad-CAM, LIME, and DeepLIFT for explainable multi-label disease classification.	Achieved $AUC \approx 0.87$ with improved interpretability, but limited by class imbalance and no lung segmentation.
2.	Comprehensive Lung Disease Detection Using Deep Learning Models with Explainable AI – Sarker et al. (2024)	arXiv	Hybrid chest X-ray datasets	Hybrid deep learning model using VGG16, DenseNet121, and ResNet50V2 with explainable AI for disease detection.	Achieved high accuracy in detecting lung diseases like COVID-19 and pneumonia, but requires high computation and lacks segmentation.
3.	Revolutionizing Lung Disease Diagnosis – Priyatharsini et al. (2024)	SSRN	Chest X-ray dataset	Hybrid deep learning model combined with explainable AI to analyze lung diseases.	Improves transparency and helps doctors interpret predictions, but the model is complex and dataset imbalance remains.
4.	CheXNet: Radiologist-Level Pneumonia Detection – Rajpurkar et al. (2017)	arXiv	Large chest X-ray dataset (100k+ images)	DenseNet-121 deep learning model used to detect pneumonia from X-ray images.	Achieved radiologist-level performance but focuses only on pneumonia and lacks explainability.
5.	Explainable Lung Disease Classification from Chest X-Ray Images – T. T. Ifty et al.	arXiv	Chest X-ray dataset	CNN model with explainable AI methods such as Grad-CAM and LIME.	Detects diseases like COVID-19, pneumonia, and tuberculosis with good accuracy and improved interpretability.

	(2024)				
6.	Explainable AI-based Lung Disease Diagnosis Using Chest X-ray Images – N. Veeramani et al. (2025)	PMC	Chest X-ray dataset	Transfer learning models VGG16, VGG19, and ResNet50 with lung segmentation and explainable AI.	Improves diagnostic accuracy and transparency, but requires high computational resources.
7.	Explainable AI Model for COVID-19, Pneumonia and Tuberculosis Detection – E. Mahamud et al. (2024)	ScienceDirect	Chest X-ray dataset	DenseNet-based transfer learning combined with explainable AI methods.	Provides accurate detection and visual explanations, but focuses on limited diseases and needs larger dataset validation.
8.	CNN-Based Explainable AI Framework for Pulmonary Disease Detection – Z. Naz et al. (2023)	MDPI	Chest X-ray dataset	CNN and ResNet-based transfer learning with explainable AI visualization.	Improves disease detection and interpretability but does not address dataset imbalance and lacks segmentation.
9.	Deep Learning for Pneumonia Detection in Chest X-ray Images – Review Paper (2024)	PMC	Various chest X-ray datasets	Review of different CNN architectures, datasets, and preprocessing techniques used for pneumonia detection.	Provides overview of recent research trends but does not propose a new model.

10.	Precise Zero-Shot Dense Retrieval without Relevance Labels – Gao, Ma, Lin, and Callan (2023)	arXiv	BEIR benchmark datasets (multiple retrieval datasets)	Proposes a zero-shot dense retrieval framework using pre-trained language models without requiring relevance labels. The method focuses on improving retrieval performance by leveraging contrastive learning and generalization across domains.	Achieved strong retrieval performance in zero-shot settings, demonstrating effectiveness without labeled data. However, the approach is limited to retrieval tasks and does not address domain-specific prediction or end-to-end application systems.
-----	--	-------	---	--	---

2.2 EXISTING SYSTEMS

Lung disease detection using chest X-ray images has evolved significantly with the advancement of deep learning and artificial intelligence techniques. Various approaches ranging from traditional machine learning models to advanced deep learning architectures have been proposed to improve diagnostic accuracy. Each method has its own strengths and limitations depending on the dataset, model complexity, and interpretability requirements. Below is a comprehensive overview of existing systems categorized based on their methodologies:

1. Traditional Machine Learning Models

Traditional machine learning techniques were initially used for medical image classification. These methods rely on handcrafted feature extraction and statistical learning, but they struggle to capture complex patterns in medical images.

➤ **Support Vector Machines (SVM):**

- Used for classification of lung diseases based on extracted features. Performs well on small datasets but struggles with high-dimensional image data.
- Requires manual feature engineering, limiting its effectiveness.

➤ **Decision Trees (DT):**

- Easy to interpret and implement.
- Prone to overfitting and less effective for complex image patterns.

➤ **Random Forest and Ensemble Methods:**

- Combine multiple models to improve accuracy.
- Computationally intensive and lack interpretability in medical applications.

2. Deep Learning Models

Deep learning models have significantly improved performance in lung disease detection by automatically extracting features from chest X-ray images.

➤ **Convolutional Neural Networks (CNNs):**

- Widely used for feature extraction and disease classification.
- Capable of learning spatial patterns in medical images.
- Lack interpretability, making them black-box models.

➤ **Transfer Learning Models (ResNet, DenseNet, VGG):**

- Use pre-trained models to improve accuracy with limited data.
- Achieve high performance in classification tasks. Do not inherently provide explanations for predictions.

➤ **Recurrent Neural Networks (RNNs):**

- Used in some hybrid approaches for sequence modeling.
- Less commonly used for static image-based tasks like X-rays.

3. Explainable AI Models

To overcome the black-box nature of deep learning models, explainable AI techniques have been introduced.

➤ **Grad-CAM and LIME:**

- Provide visual explanations by highlighting important regions in X-ray images. Improve transparency and trust in predictions.
- Do not improve classification accuracy directly.

4. Hybrid Models

Recent approaches combine multiple techniques to improve performance and interpretability.

➤ **CNN + Explainability Models:**

- Combine CNNs with Grad-CAM or LIME for better interpretation.
- Improve trust but still lack anatomical localization.

➤ **CNN + Segmentation Models:**

- Use U-Net for lung segmentation along with classification models.
- Improve focus on relevant lung regions and accuracy.

➤ **Multi-Model Hybrid Systems:**

- Combine multiple architectures like ResNet, EfficientNet, and segmentation models.
- Provide better performance but increase computational complexity.

2.3 DISADVANTAGES OF EXISTING SYSTEMS

➤ **Computational Complexity:**

- Advanced deep learning and hybrid models require high computational resources and longer training times.
- Integration of multiple models increases system complexity.

➤ **Lack of Interpretability:**

- Many deep learning models function as black boxes, making it difficult to understand how predictions are made.
- Limited trust in clinical applications.

➤ **Limited Anatomical Focus:**

- several models do not include lung segmentation, leading to analysis of irrelevant regions. Reduces accuracy and reliability.

➤ **Data Dependency and Imbalance:**

- Performance depends heavily on large and well-balanced datasets. Class imbalance affects detection of rare diseases.

➤ **Lack of User Interaction:**

- Most existing systems only provide predictions without explanations or user guidance. No support for answering medical queries.

2.4 PROPOSED SYSTEM

To overcome the limitations of existing lung disease detection systems, the proposed system introduces an **explainable and modular deep learning framework** for automated analysis of chest X-ray images, integrated with a **Retrieval-Augmented Generation (RAG)–based medical chatbot**. The system combines the strengths of convolutional neural networks for classification, segmentation models for anatomical localization, and explainable AI techniques to improve transparency and trust in predictions.

The proposed system leverages multiple components, each contributing to a specific aspect of the diagnosis process:

- **Convolutional Neural Networks (ResNet-101 & EfficientNet-B0):** These models are used for **multi-label classification** of chest X-ray images. They extract deep hierarchical features and identify the presence of multiple thoracic diseases. The ensemble of ResNet and Efficient Net improves robustness and overall prediction performance.
- **U-Net (Lung Segmentation Model):** The U-Net model is employed to **segment lung regions** from chest X-ray images, providing anatomical context through segmentation visualization displayed alongside predictions.
- **Grad-CAM (Explainability Module):** Grad-CAM is applied to generate **visual heatmaps** that highlight the regions of the X-ray image influencing the model's predictions. This enhances interpretability and allows medical professionals to better understand and trust the system's output.
- **Prediction Refinement (Thresholding Mechanism):** Thresholding techniques are used to convert raw probability outputs into final predictions. This helps in **reducing false positives** and improving the overall reliability of the system.
- **RAG-Based Medical Chatbot**

The system integrates a Retrieval-Augmented Generation (RAG) pipeline that combines:

- Embedding Model for query representation
- FAISS Vector Database for retrieving relevant medical documents
- LLaMA3 (via Ollama) for generating structured responses

This module enables users to ask medical questions and receive context-aware, reliable answers with references, improving user interaction and understanding.

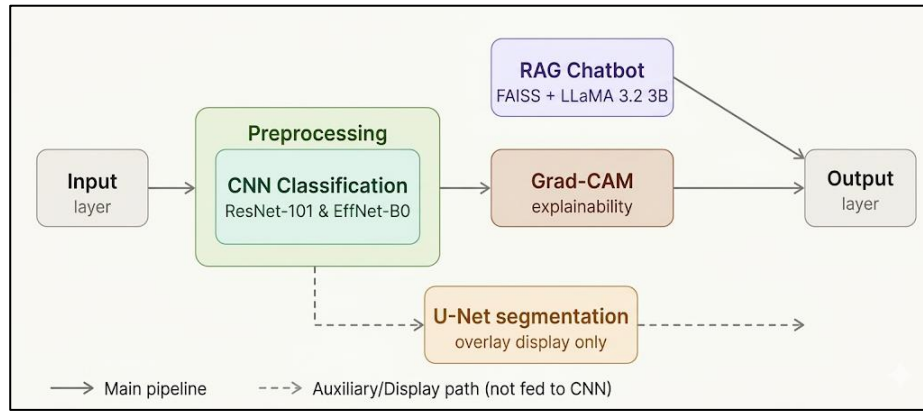


Fig 2.1 Hybrid Model Architecture

Fig 2.1 shows the architecture of the proposed **Explainable Lung Disease Detection System integrated with a RAG-based chatbot**. The model begins with an **input layer**, where a chest X-ray image is provided by the user. The image is first passed through a **preprocessing block**, which performs resizing, normalization, and formatting to make it suitable for deep learning models.

The pre-processed image is then fed into the **CNN-based classification block**, consisting of models such as **ResNet-101** and **EfficientNet-B0**, which extract deep features and perform multi-label disease prediction. In parallel, the image is processed through the **U-Net segmentation block**, which generates a lung mask by identifying the relevant lung regions and removing unnecessary background information.

The extracted features and predictions are then passed through a **thresholding layer**, which refines the results by reducing false positives and selecting the most probable diseases. To enhance interpretability, the refined outputs are processed by the **Grad-CAM layer**, which generates heatmaps highlighting the regions of the X-ray image that significantly influence the model's predictions.

Finally, the system produces results in the **output layer**, which include detected diseases, confidence scores, lung segmentation masks, and Grad-CAM visualizations. In addition to the image diagnosis pipeline, the system incorporates a **RAG-based chatbot module**, where user queries are converted into embeddings, relevant medical documents are retrieved using a FAISS vector database, and a language model generates context-aware responses.

This integrated approach combines CNN-based classification for feature extraction, U-Net for anatomical localization, Grad-CAM for explainability, and RAG for interactive assistance, ensuring improved accuracy, interpretability, and user engagement in lung disease detection.

Advantages of our model

- **Improved Detection Accuracy:** The combination of **ResNet-101** and **EfficientNet-B0** enables effective feature extraction and multi-label classification, leading to higher accuracy compared to single-model approaches.
- **Better Focus on Relevant Regions:** The **U-Net segmentation model** isolates lung regions, removing unnecessary background noise and improving the reliability of predictions.
- **Enhanced Interpretability:** The use of **Grad-CAM** provides visual heatmaps that highlight important regions influencing the model's decisions, increasing transparency and trust.
- **Reduced False Positives:** The **thresholding mechanism** refines model outputs by selecting only significant predictions, improving overall precision and reliability.
- **Interactive User Support:** The integration of a **RAG-based chatbot** allows users to ask questions and receive context-aware medical explanations, enhancing user engagement and understanding.
- **Modular and Scalable Architecture:** The system is designed with independent modules (classification, segmentation, explainability, and RAG), making it easy to scale, maintain, and extend.

2.5 CONCLUSION

In this project, an explainable and intelligent system for lung disease detection from chest X-ray images has been developed by integrating deep learning and natural language processing techniques. The use of CNN-based models such as ResNet-101 and EfficientNet-B0 enables accurate multi-label disease classification, while the U-Net model improves anatomical understanding through lung segmentation. The incorporation of Grad-CAM enhances interpretability by providing visual explanations of predictions, increasing trust in the system. Additionally, the integration of a RAG-based chatbot allows users to obtain context-aware medical information.

3. REQUIREMENT ANALYSIS

3.1 INTRODUCTION

Requirement analysis is a crucial phase in software development that involves identifying and documenting both functional and non-functional requirements of the system. For the **Lung Disease Detection and RAG-based Chatbot system**, this phase focuses on understanding user needs, defining system functionalities, and identifying the necessary software and hardware resources. It ensures smooth implementation, efficient model deployment, and effective interaction between the disease detection module and the chatbot system.

3.2 SOFTWARE REQUIREMENTS SPECIFICATIONS

3.2.1 User Requirements:

User Authentication: Users should be able to register and log in securely.

Image Upload: Users should be able to upload chest X-ray images for disease detection.

Disease Prediction: The system should analyse the uploaded image and display predicted lung diseases with confidence scores.

Visualization: The system should display Grad-CAM heatmaps and lung segmentation results for better understanding.

Chatbot Interaction: Users should be able to ask medical-related questions and receive context-aware responses.

User-Friendly Interface: The system should provide easy navigation between image upload, results, and chatbot interface.

3.2.2 Software Requirements:

Development Environment: IDE/Editor: Visual Studio Code (VS Code) – Used for integrating backend, frontend, and API components.

Dataset Source: Kaggle – Used for collecting and training chest X-ray image datasets.

Model Training Platform: Google Colab – Used for training deep learning models using GPU

Backend Framework: FastAPI – Used to handle API requests and integrate different modules.

Programming Language: Python 3.x – Used for implementing deep learning models

Deep Learning Libraries: TensorFlow, Keras – Used for building and training CNN and U-Net

Data Processing Libraries: NumPy, Pandas – Used for data manipulation and preprocessing.

Visualization Libraries: Matplotlib, Seaborn – Used for plotting and result visualization.

Explainability Tools: Grad-CAM – Used for generating visual explanations.

RAG Framework: Ollama – Used to run LLaMA3 model locally for response generation.

Vector Database: FAISS – Used for storing and retrieving embeddings for the chatbot.

Web Browser: Google Chrome / Mozilla Firefox – For accessing the application.

Table 2: Packages Used

S.No	Package Name	Description
1	FastAPI	Lightweight web framework for building APIs and handling requests
2	NumPy	Supports numerical computations and array operations
3	Pandas	Used for data manipulation and preprocessing
4	Matplotlib	Used for plotting graphs and visualizations
5	Seaborn	Provides advanced statistical visualizations
6	TensorFlow	Used for building and training deep learning models used for loading resnet101, efficientnet b0, unet models
7	Scikit-learn	Provides tools for data preprocessing, model evaluation, and machine learning tasks.
8	FAISS	Efficient similarity search vector embeddings in RAG
9	Ollama	Used to run LLAM3 model for chatbot responses

3.2.3 Hardware Requirements:

Operating System: Windows 10/11 (64-bit) or Linux

Processor: Intel Core i5 / i7 or equivalent

RAM: Minimum 8 GB (16 GB recommended for smooth performance)

Storage: 512 GB or higher

GPU (Optional): Required for faster model training (Google Colab provides GPU support)

Table 3: Hardware Requirements

Hardware Requirements	Used Version
Processor	Intel Core i5/i7-1135G7 CPU 2.40GHz
RAM	8.00-16.00 GB
Hard Disk	512-1 TB

3.2.4 Non-Functional Requirements:

- **Performance:** The system should provide predictions and chatbot responses within a few seconds.
- **Scalability:** The system should handle increasing amounts of data and concurrent users efficiently.
- **Security:** User data should be encrypted, and the system should follow secure authentication and authorization protocols.
- **Usability:** The interface should be simple, intuitive, and easy to use for all users.

3.3 CONTENT DIAGRAM OF THE PROJECT

The content diagram provides an overview of the system architecture and workflow of the proposed **Lung Disease Detection and RAG-based Chatbot system**. It illustrates the complete process, starting from user interaction to disease prediction and medical query response generation.

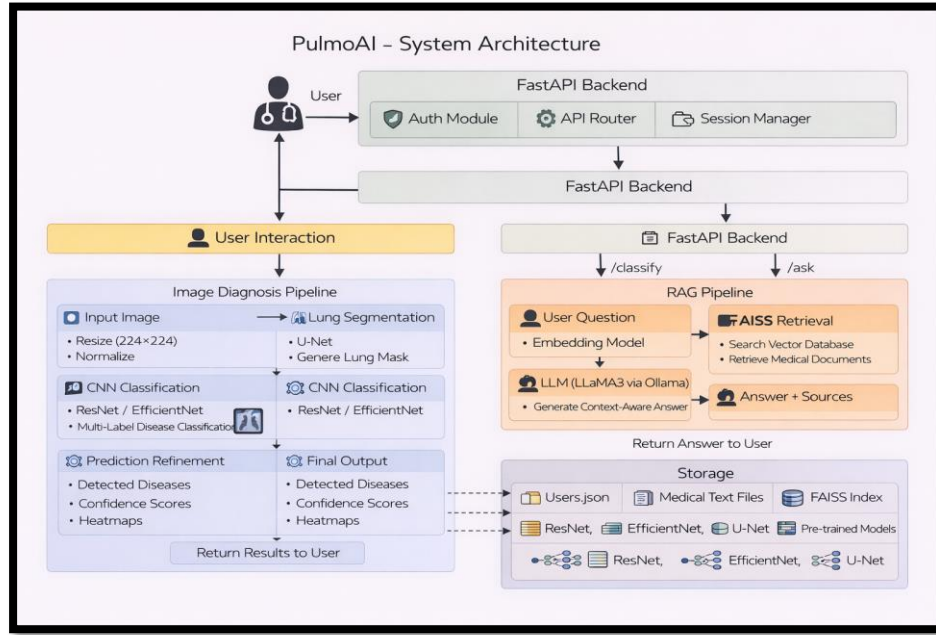


Fig 3.1 System Architecture

Fig 3.1 represents the system architecture of the proposed project, showing the integration of deep learning models with an intelligent chatbot system. The process begins with the user uploading a chest X-ray image and optionally asking a medical-related query. The input is processed through the FastAPI backend, which manages request handling, authentication, and communication between system components.

The uploaded image is first passed through the image acquisition and preprocessing stage, where operations such as resizing, normalization, and formatting are performed to prepare the image for model input. The pre-processed image is then forwarded to the CNN-based classification pipeline, which utilizes models such as ResNet-101 and EfficientNet-B0 to extract features and predict multiple lung diseases.

To enhance interpretability, the system applies the Grad-CAM explainability layer, which produces heatmaps highlighting important regions in the X-ray image that influence the prediction. These visual explanations assist users and medical professionals in understanding the model's decisions.

In parallel, the system includes a RAG (Retrieval-Augmented Generation) pipeline for handling user queries. The query is converted into embeddings using an embedding model, and relevant medical documents are retrieved from the FAISS vector database. The retrieved context is then passed to a language model (LLaMA3 via Ollama), which generates accurate and context-aware responses.

Finally, the system outputs include predicted diseases, confidence scores, segmentation masks, Grad-CAM visualizations, and chatbot responses. All data, including user information, medical text files, and FAISS indexes, are managed through the backend for efficient storage and retrieval.

Thus, the proposed system integrates image processing, deep learning, explainable AI, and natural language processing into a unified framework, ensuring accurate, interpretable, and interactive healthcare assistance.

3.4 ALGORITHMS AND FLOWCHARTS

3.4.1 ALGORITHMS

We propose an advanced deep learning-based system for lung disease detection and medical query assistance, which integrates multiple models including ResNet-101, EfficientNet-B0, U-Net, Grad-CAM, and a RAG-based chatbot. The system is designed to improve prediction accuracy, interpretability, and user interaction.

During the training phase, the classification models (ResNet-101 and EfficientNet-B0) are trained using the NIH Chest X-ray dataset, which contains multiple thoracic disease labels. These models learn to extract deep features and perform multi-label classification. During validation, appropriate thresholding techniques are applied to refine prediction probabilities and reduce false positives.

The final prediction is obtained using an ensemble approach, where outputs from both models are combined with a threshold value of 0.5, ensuring improved accuracy and robustness compared to individual models. For anatomical localization, a U-Net segmentation model is trained on the JSRT dataset to extract lung regions. This model is then applied to NIH images during testing to focus on relevant areas

To enhance interpretability, Grad-CAM is used as a post-processing technique (without training) to generate heatmaps that highlight important regions influencing the model's predictions.

In addition to image-based diagnosis, the system integrates a RAG (Retrieval-Augmented Generation) pipeline. The chatbot is powered by Ollama (LLaMA3) and uses customized medical

text files covering 14 lung diseases. User queries are converted into embeddings, relevant documents are retrieved using FAISS, and context-aware responses are generated.

Thus, the proposed system combines deep learning, explainable AI, and natural language processing to provide an accurate, interpretable, and interactive solution for lung disease detection.

Key Advantages:

Improved Prediction Accuracy: Ensemble of ResNet-101 and EfficientNet-B0 improves classification performance.

Better Localization: U-Net segmentation focuses on lung regions, improving reliability.

Enhanced Interpretability: Grad-CAM provides visual explanations without requiring additional training.

Reduced False Positives: Thresholding during validation improves prediction quality.

Interactive Assistance: RAG chatbot provides context-aware medical responses using domain-specific data.

3.4.2 FLOWCHARTS

A. FLOWCHART FOR MODEL TRAINING AND PREDICTION

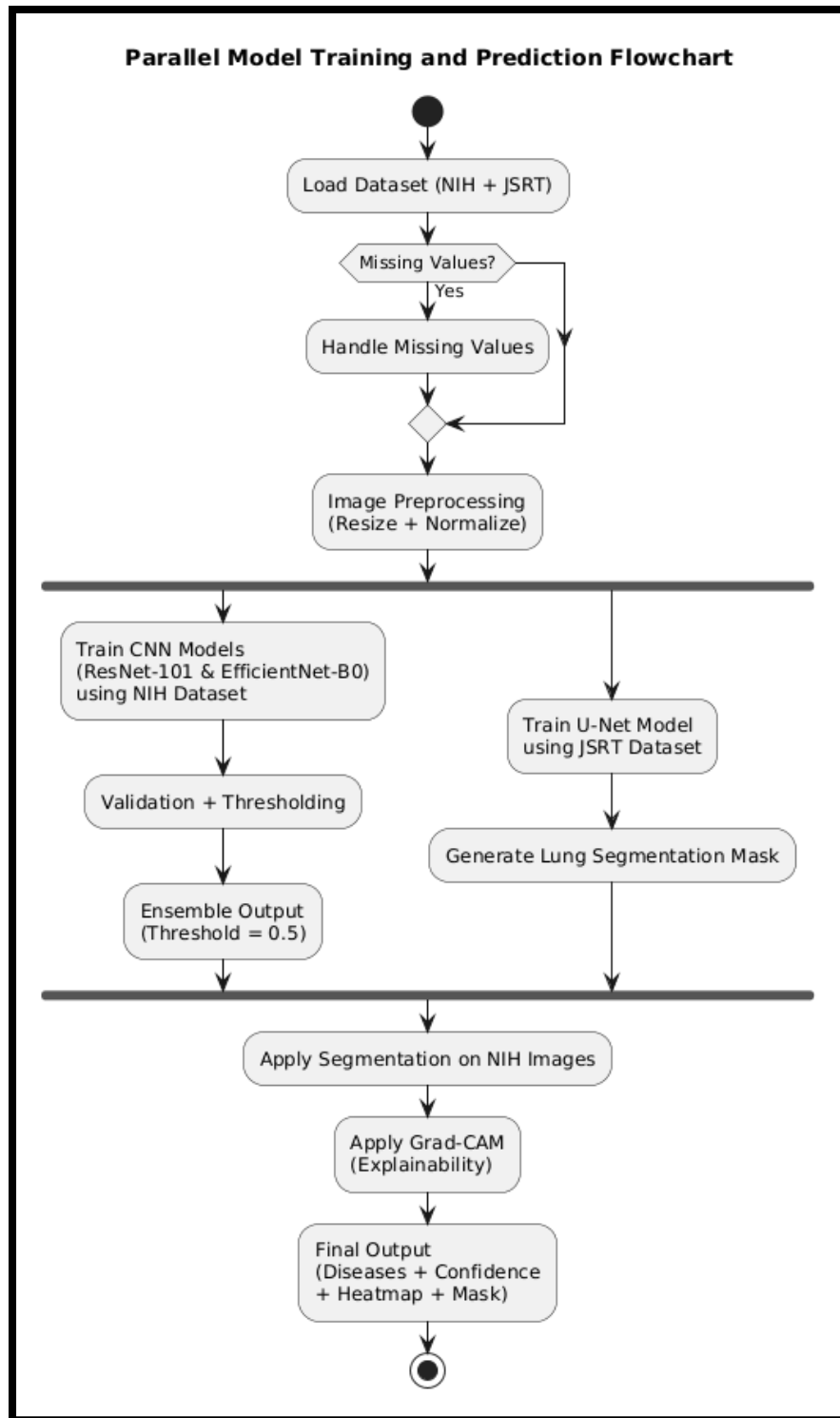


Fig 3.2 Flowchart for model training and prediction

Fig 3.2 depicts the flowchart of the model training and prediction process. The process begins with loading the NIH chest X-ray dataset, followed by preprocessing steps such as normalization and resizing. The dataset is then used to train deep learning models including ResNet-101 and EfficientNet-B0.

During validation, thresholding is applied to refine predictions. The outputs from both models are combined using an ensemble method (threshold = 0.5) to generate final predictions. In parallel, lung segmentation is performed using a U-Net model trained on the JSRT dataset. The segmentation mask is applied to NIH images to focus on lung regions.

Grad-CAM is then used to generate heatmaps for explainability. Finally, the system outputs predicted diseases along with confidence scores and visual explanations.

B. FLOWCHART FOR RAG -BASED CHATBOT

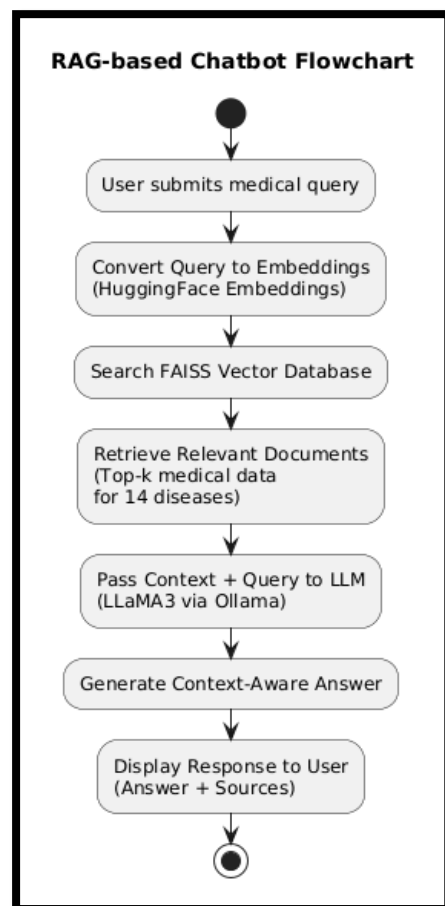


Fig 3.3 Flowchart for RAG-based chatbot

Fig 3.3 illustrates the flow of the RAG-based chatbot. The process begins when a user submits a medical query. The query is converted into embeddings using a Hugging Face embedding model. These embeddings are used to search the FAISS vector database, which contains preprocessed medical documents related to 14 lung diseases. The most relevant documents are retrieved and passed along with the query to the LLaMA3 model via Ollama. The model generates a context-aware response which is then displayed to user along relevant sources.

3.5 CONCLUSION

The requirement analysis phase of the proposed Lung Disease Detection and RAG-based Chatbot system successfully identifies and documents the functional and non-functional requirements necessary for effective system development. The software and hardware specifications, along with user requirements, ensure smooth implementation, efficient model training, and seamless integration of different system components.

The content diagram and flowcharts provide a clear visualization of the system architecture, highlighting key stages such as data preprocessing, parallel model training using ResNet-101, EfficientNet-B0, and U-Net, prediction refinement through thresholding and ensemble methods, and explainability using Grad-CAM. Additionally, the integration of a RAG-based chatbot using FAISS and LLaMA3 enables context-aware medical query handling, enhancing user interaction and understanding.

Overall, the system is designed to deliver accurate, interpretable, and interactive results while maintaining scalability, usability, and performance. This phase ensures that the project is well-structured and ready for implementation, providing a strong foundation for developing an intelligent healthcare assistance system.

4.DESIGN

4.1 INTRODUCTION

In software development, the design phase plays a crucial role in transforming system requirements into a structured and implementable architecture. It provides a clear blueprint that guides developers in building an efficient, scalable, and well-organized system. A well-designed system not only improves clarity and maintainability but also reduces the chances of errors and rework during later stages of development.

The design phase of the proposed **Lung Disease Detection and RAG-based Chatbot system (PulmoAI)** focuses on integrating deep learning models with an interactive backend to provide accurate and interpretable medical assistance. The system combines CNN-based classification models (ResNet-101 and EfficientNet-B0), a U-Net segmentation model, and a Grad-CAM explainability module with a FastAPI-based backend. Additionally, a Retrieval-Augmented Generation (RAG) module using FAISS and LLaMA3 via Ollama is incorporated to provide context-aware medical responses.

To represent the system's behaviour and structure, various UML diagrams are used. These diagrams help visualize how different modules interact, how data flows through the system, and how user requests are processed. A modular design approach is adopted to ensure scalability, flexibility, and ease of maintenance, allowing future enhancements such as adding new diseases or improving model performance.

UML Relationships

UML diagrams depict the relationships between various entities, helping to understand the structural and behavioral aspects of the system. The key types of relationships in UML are:

- **Association:** Represents interaction between system components such as user and system modules.
- **Dependency:** Indicates that one module depends on another for functionality, such as prediction depending on trained models.
- **Aggregation:** Shows a “has-a” relationship, where one module contains another, such as the system containing multiple processing modules
- **Generalization:** Describes a parent-child relationship between different entities.
- **Realization:** Specifies a relationship where one entity defines behavior or responsibility, and another carries it out.

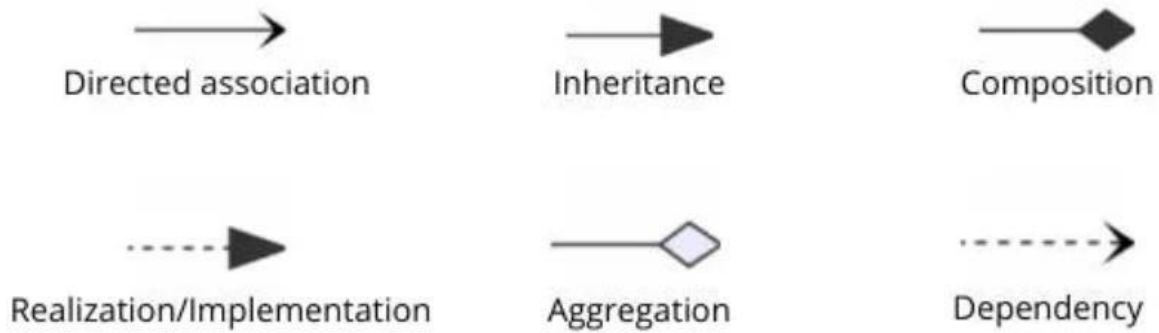


Fig 4.1 Relationship Diagram

The relationships described above are visually represented in **Fig 4.1 Relationships**, which shows various UML relationship types such as one-way and two-way associations, inheritance, aggregation, dependency, and realization.

4.2 UML DIAGRAMS

4.2.1 USE CASE DIAGRAM

The **Use Case Diagram** provides a high-level visual representation of the interactions between the **User** and the **System** in the **Lung Disease Detection and RAG-based Chatbot**. It highlights the key functionalities available to the user and how the system processes their requests. It consists of the following artifacts: Actor, Use case, Secondary Actor and Use Case Boundary.

- **Actor:** A role of a user that interacts with the system you're modeling is represented by an actor. A person, an organization, a machine, or another external system can all be considered as users.
- **Use-Case:** A use case is a function that a system does to help the user achieve their goal. A use case must produce an observable result that is useful to the system's user.
- **Secondary Actor:** Actors who are not directly interacting with the system are secondary actors and are placed on the right side of the use case boundary.
- **Use Case Boundary:** It is the boundary to separate the use cases that are internal to the system from the actors that are external to the system.

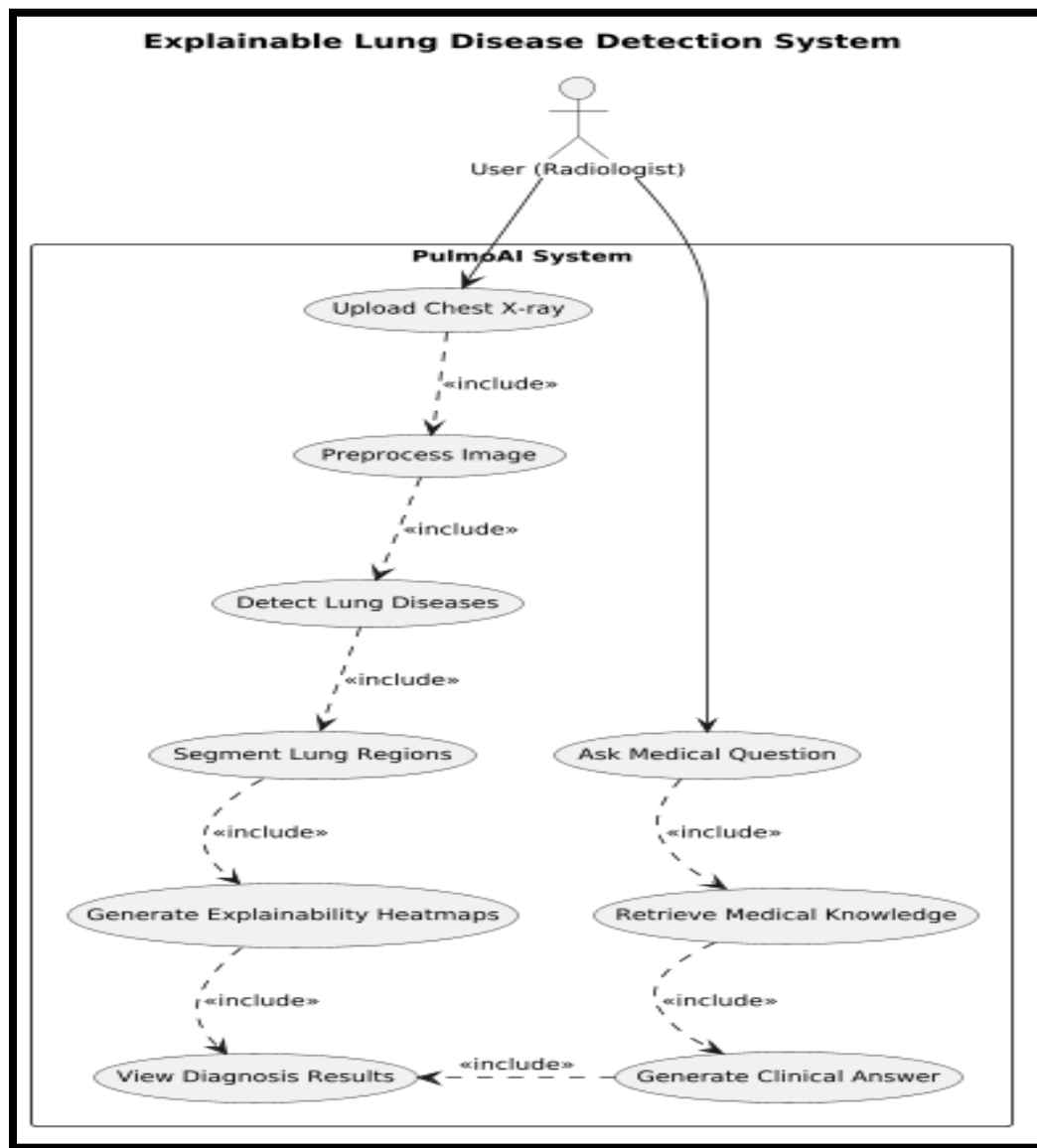


Fig 4.2 Use Case Diagram

Fig 4.2 represents the **Use Case Diagram** for the lung disease detection

Actors:

- **User:** The primary actor who interacts with the system by uploading X-ray images and asking medical queries.
- **System:** The backend system responsible for processing images, generating predictions, and handling chatbot responses.

Use Cases:

- **Register/Login:** User creates an account or logs into the system.
- **Upload X-ray Image:** User uploads chest X-ray for analysis.
- **Predict Disease:** System processes the image and predicts lung diseases.
- **View Results:** Displays diseases, confidence scores, segmentation masks, and heatmaps.
- **Ask Medical Query:** User interacts with the chatbot for additional medical information.
- **View Previous Results:** User can access previously analysed data.

4.2.2 CLASS DIAGRAM

Class diagram is a static representation of the system that highlights the structure, attributes, and relationships between different classes. It helps in understanding how various components of the system interact and how responsibilities are distributed among them. The class diagram provides a clear blueprint for implementing the system in an object-oriented manner.

Artifacts of the Class Diagram are:

- **Class:** The class represents similar objects and consists of name, responsibilities and attributes.
- **Name:** Name of the class is the identity of the class.
- **Responsibilities:** Responsibilities represents the duties to be done or functionalities that the class exhibits.
- **Attributes:** Attributes represent the properties that a particular class has and the objects of the class will have similar attributes.
- **Methods (operations):** methods define the actions or functionalities performed by a class .These include operations such as preprocessing images, predicting diseases, generating embeddings or retrieving data.

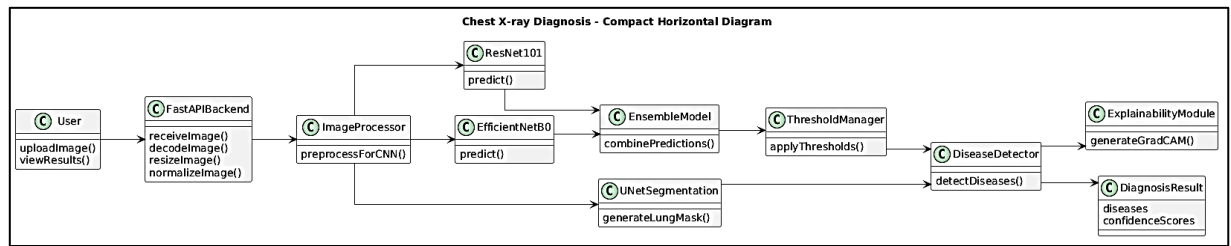


Fig 4.3.1 Class Diagram of chest x-ray diagnosis

Fig 4.3.1 represents the Class Diagram for the **Lung Disease Detection System**. It illustrates the interaction between different components involved in image processing, disease prediction, segmentation, and explainability.

User Class: Handles user interactions such as uploading X-ray images and viewing diagnosis results.

FastAPI Backend Class: Acts as the central controller that receives user inputs, performs preprocessing operations such as resizing and normalization, and coordinates communication between different modules.

ImageProcessor Class: Responsible for preprocessing input images to make them suitable for model input.

ResNet101Model Class & EfficientNetB0Model Class: These classes perform disease prediction using deep learning models. They extract features and generate prediction probabilities for multiple lung diseases.

EnsembleModel Class: Combines predictions from both ResNet-101 and EfficientNet-B0 models to improve overall accuracy.

ThresholdManager Class: Applies thresholding techniques to refine predictions and reduce false positives.

UNetSegmentation Class: Generates lung masks by segmenting the lung regions from the X-ray images.

DiseaseDetector Class: Integrates outputs from classification and segmentation modules to detect diseases effectively.

ExplainabilityModule Class: Generates Grad-CAM heatmaps to provide visual explanations for model predictions.

DiagnosisResult Class: Stores final output including detected diseases and confidence scores, which are displayed to the user.

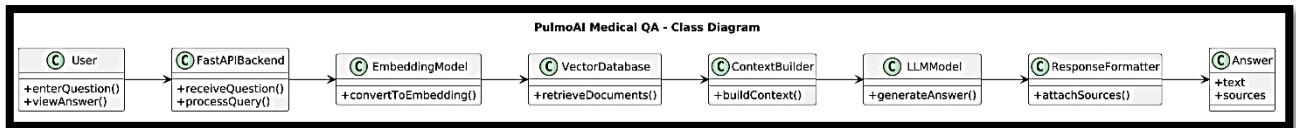


Fig 4.3.2 Class Diagram of RAG-chatbot

Fig 4.3.2 represents the Class Diagram for the RAG-based Medical Chatbot system. It shows how user queries are processed and how responses are generated using retrieval and language models.

User Class: Allows users to enter medical queries and view chatbot responses.

FastAPIBackend Class: Handles incoming queries and manages the interaction between different modules in the chatbot pipeline.

EmbeddingModel Class: Converts user queries into vector representations (embeddings) for efficient similarity search.

VectorDatabase (FAISS) Class: Stores medical documents and retrieves relevant information based on query embeddings.

ContextBuilder Class: Builds meaningful context by combining retrieved documents with the user query.

LLMModel Class: Generates responses using the LLaMA3 model via Ollama based on the provided context.

ResponseFormatter Class: Formats the generated response and attaches relevant sources for better understanding.

Answer Class: Stores the final chatbot response along with supporting references and displays it to the user.

4.2.3 SEQUENCE DIAGRAM

The sequence diagram, also known as an event diagram, depicts the flow of messages through the system. It aids in the visualization of a variety of dynamic scenarios. It depicts communication between any two lifelines as a time-ordered series of events, as if these lifelines were present at the same moment. The message flow is represented by a vertical dotted line that extends across the bottom of the page in UML whereas the lifeline is represented by a vertical bar. It encompasses both iterations and branching.

Notations of Sequence Diagram are:

Lifeline: A lifeline represents an individual participant in the sequence diagram. It is at the very top of the diagram.

Actor: An actor is a character who interacts with the subject and plays a part. It isn't covered by the system's capabilities. It depicts a role in which human users interact with external devices or subjects.

Message: Message denotes the interactions between the messages. They are in the sequential order in the timeline.

Call message: By defining a specific communication between the interaction's lifelines, it shows that the target lifeline has invoked an operation.

Return Message: It specifies a specific communication between the interaction lifelines, which reflect the flow of data from the receiver of the associated caller message.

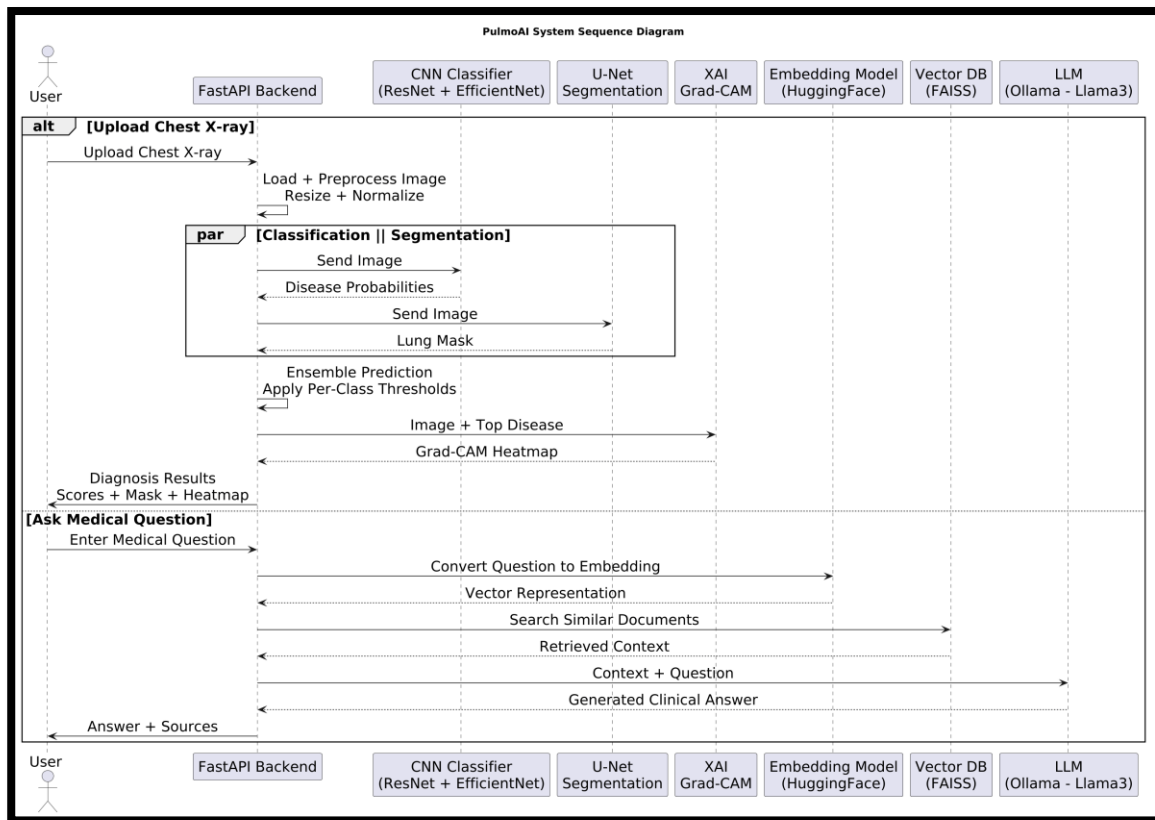


Fig 4.4 Sequence Diagram

Fig 4.4 represents the Sequence Diagram for the proposed **Lung Disease Detection and RAG-based Chatbot system (PulmoAI)**. It shows the interaction between the user, backend, deep learning models, and the RAG pipeline.

User Interaction and Image Diagnosis:

- The user uploads a chest X-ray image to the system.
- The request is received by the **FastAPI Backend**, which performs preprocessing operations such as resizing and normalization.
- The preprocessed image is then sent in parallel to:
- **CNN Classifier (ResNet-101 & EfficientNet-B0)** for disease prediction
- **U-Net Segmentation module** for generating lung masks
- The CNN model returns disease probabilities, while the U-Net model returns the lung segmentation mask.
- The backend performs **ensemble prediction and thresholding** to refine results.
- The processed image is passed to the **Grad-CAM module**, which generates heatmaps for explainability.
- Finally, the system returns the **diagnosis results**, including detected diseases, confidence scores, lung mask, and heatmap, to the user.

Medical Query Processing (RAG Pipeline):

- The user submits a medical question through the interface.
- The query is sent to the **FastAPI Backend**, which forwards it to the embedding module.
- The **Embedding Model (Hugging Face)** converts the query into vector representation.
- The vector is used to search the **FAISS Vector Database**, retrieving relevant medical documents.
- The retrieved context is combined with the user query and sent to the **LLM (LLaMA3 via Ollama)**.
- The LLM generates a context-aware response based on the provided information.
- The final answer, along with relevant sources, is returned to the user.

4.3 MODULAR DESIGN AND ORGANIZATION

The modular design of the proposed **Lung Disease Detection and RAG-based Chatbot system (PulmoAI)** ensures a well-structured and scalable architecture. The system is divided into multiple modules, each responsible for specific functionalities, enabling efficient processing, easy maintenance, and future enhancements.

Backend Module (FastAPI):

Acts as the central controller that receives user requests, performs preprocessing, and manages communication between different system components.

Image Processing Module:

Responsible for preprocessing operations such as decoding, resizing, and normalization of input X-ray images before feeding them into models.

Prediction Module (CNN Models):

Uses deep learning models such as ResNet-101 and EfficientNet-B0 to predict lung diseases from X-ray images.

Segmentation Module (U-Net):

Extracts lung regions by generating segmentation masks, ensuring the model focuses on relevant areas.

Explainability Module (Grad-CAM):

Provides visual explanations by generating heatmaps that highlight important regions influencing predictions.

RAG Module:

Handles medical queries using embeddings, FAISS vector database, and LLaMA3 via Ollama to generate context-aware responses.

Data Management Module:

Stores user data, prediction results, medical text files, and FAISS indexes for efficient retrieval.

System Workflow

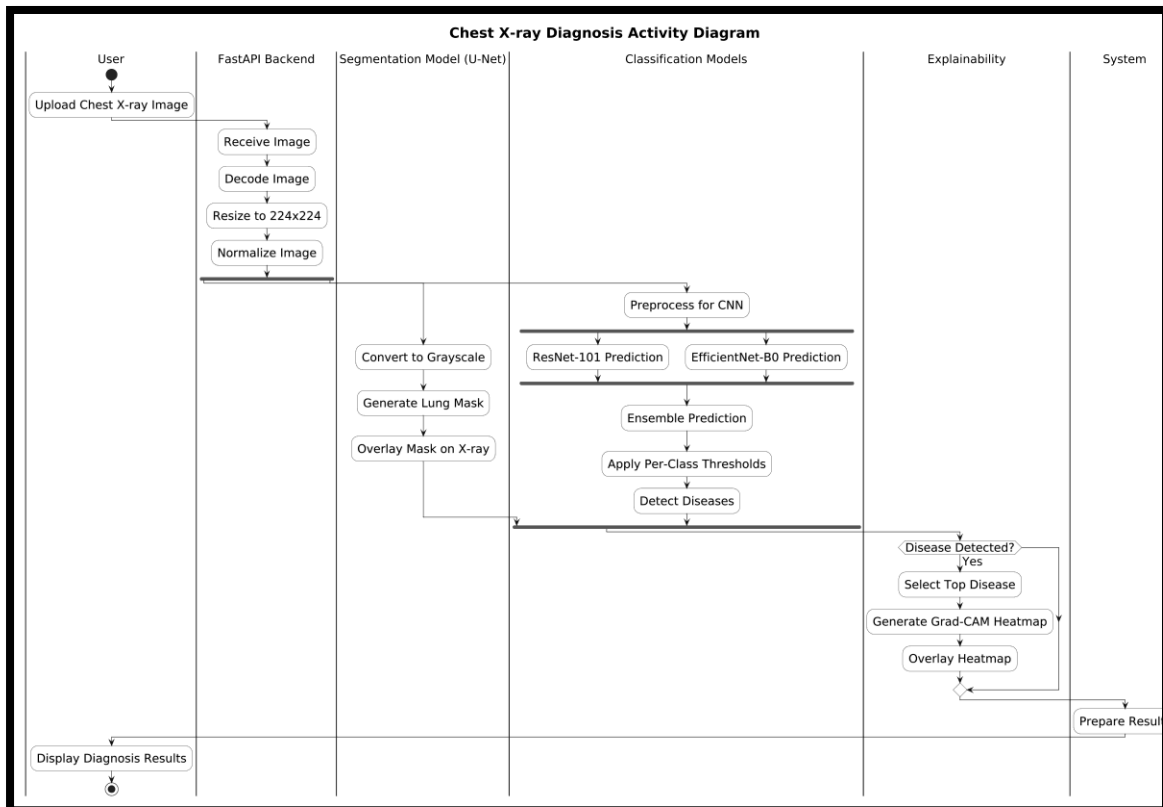


Fig 4.5.1 System Workflow of Chest X-ray Diagnosis

Fig 4.5.1 illustrates the overall workflow of the proposed system. The process begins when the user uploads a chest X-ray image. The image is received by the backend and undergoes preprocessing such as resizing and normalization.

The system then processes the image in parallel using CNN models for classification and U-Net for segmentation. The outputs are combined using ensemble techniques and thresholding to generate accurate predictions. Grad-CAM is applied to produce heatmaps for explainability.

In parallel, the system also supports medical query processing through a RAG pipeline. The user's query is converted into embeddings, relevant documents are retrieved from FAISS, and the LLaMA3 model generates a context-aware response.

Finally, the system displays the results, including predicted diseases, confidence scores, segmentation masks, heatmaps, and chatbot responses, providing a complete and interactive experience.

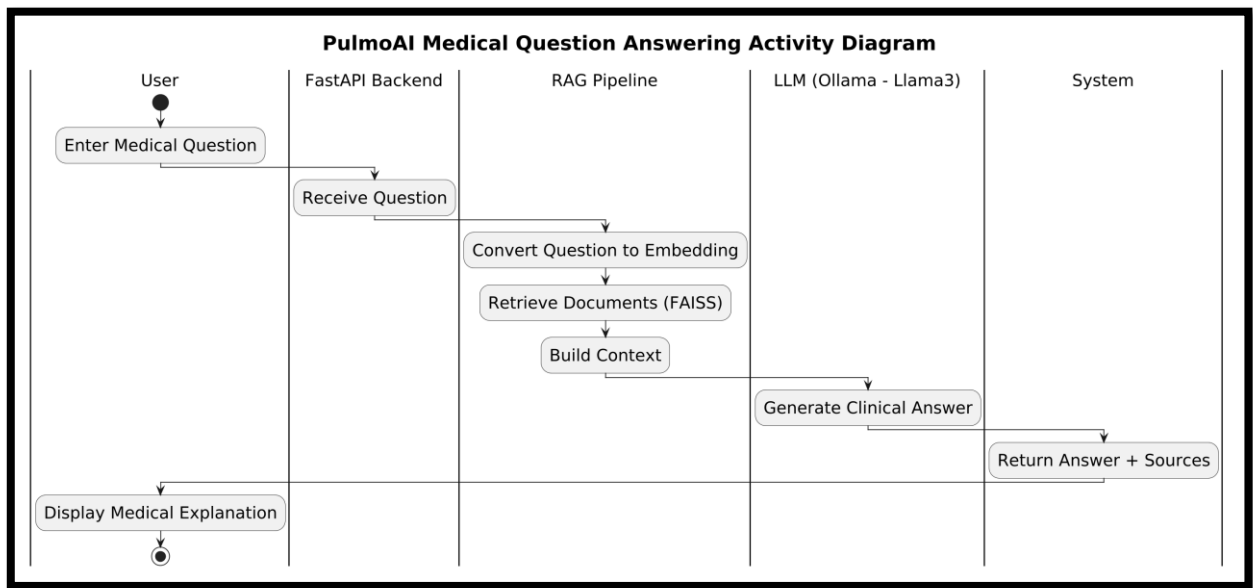


Fig 4.5.2 System Workflow of RAG-chatbot

Fig 4.5.2 illustrates the activity diagram for the Chest X-ray diagnosis process. The workflow begins with the user uploading a chest X-ray image. The backend receives the image and performs preprocessing steps such as decoding, resizing, and normalization.

The processed image is then handled in parallel by the classification models and segmentation module. The classification models generate disease probabilities, while the U-Net model produces the lung mask. These outputs are combined using ensemble prediction and thresholding.

If a disease is detected, the system selects the most relevant disease and applies Grad-CAM to generate heatmaps. The final results, including diseases, confidence scores, segmentation masks, and heatmaps, are prepared and displayed to the user.

4.4 CONCLUSION

The design phase of the proposed PulmoAI system establishes a strong and efficient architecture by integrating deep learning models, explainable AI techniques, and a RAG-based chatbot within a modular framework. The activity diagrams clearly illustrate the step-by-step workflow of both the disease detection and medical query processing pipelines.

The modular design ensures scalability, maintainability, and flexibility, allowing the system to adapt to future enhancements such as adding new diseases or improving model performance. The integration of classification, segmentation, explainability, and natural language processing provides a comprehensive and user-friendly healthcare solution.

Overall, the system is designed to be accurate, interpretable, and interactive, making it suitable for real-world medical assistance applications.

5. IMPLEMENTATION AND RESULTS

5.4 INTRODUCTION

This section presents the implementation and results of the proposed **PulmoAI system for lung disease detection and medical question answering**. The system integrates deep learning models for image-based diagnosis along with a Retrieval-Augmented Generation (RAG) pipeline for medical query handling.

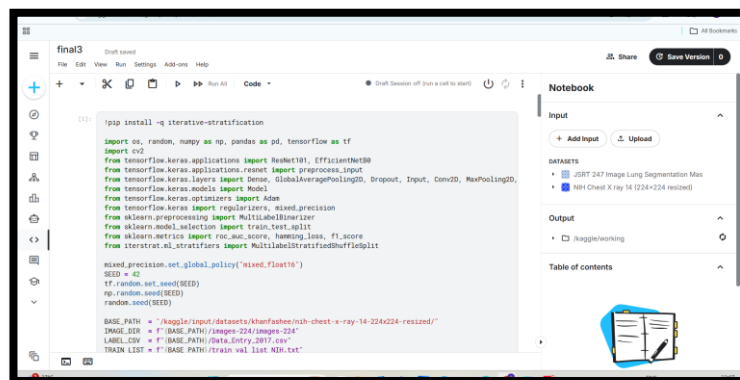
The implementation was carried out using **Python** with deep learning frameworks such as **TensorFlow and Keras**. Model training was performed on the **NIH Chest X-ray dataset**, while lung segmentation was trained using the **JSRT dataset**. The system was developed and integrated using **Visual Studio Code**, with model experimentation conducted on **Kaggle** and testing on **Google Colab**.

The implementation consists of three major components: image segmentation using U-Net, disease classification using CNN models (ResNet-101 and EfficientNet-B0), and a RAG-based chatbot for medical assistance.

Kaggle Platform:

Kaggle is a widely used cloud-based platform for data science and machine learning that provides access to datasets, computational resources, and collaborative tools. It offers a powerful environment with built-in support for Python, TensorFlow, and other libraries, along with free GPU acceleration, making it suitable for training deep learning models efficiently.

In this project, Kaggle was used for training the deep learning models, including the **ResNet-101 and EfficientNet-B0 classification models** as well as the **U-Net segmentation model**. The availability of GPU resources enabled faster training and experimentation on large-scale datasets such as the NIH Chest X-ray dataset and JSRT dataset. Kaggle notebooks also facilitated easy data handling, preprocessing, and model evaluation within a unified environment.

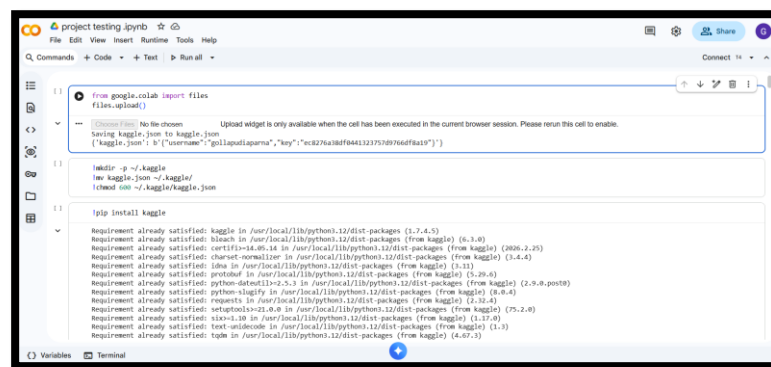


Screen 1: Kaggle interface

Google Colab Platform:

Google Colab is a cloud-based development environment that allows users to write and execute Python code in a notebook interface. It provides free access to computational resources such as GPUs and TPUs, making it suitable for running and testing machine learning models without requiring local hardware setup.

In this project, Google Colab was used for testing and evaluating the trained models. After training the models on Kaggle, they were loaded into the Colab environment for performing inference on test images. The platform enabled efficient experimentation, visualization of results, and validation of model performance. It also supported seamless integration with libraries such as TensorFlow, OpenCV, and Matplotlib, which were used for prediction, preprocessing, and result visualization.



Screen 2: colab interface

Visual Studio Code:

Visual Studio Code (VS Code) is a lightweight yet powerful code editor that supports multiple programming languages and frameworks. It provides features such as an integrated terminal, debugging tools, extension support, and version control, making it suitable for full-stack application development.

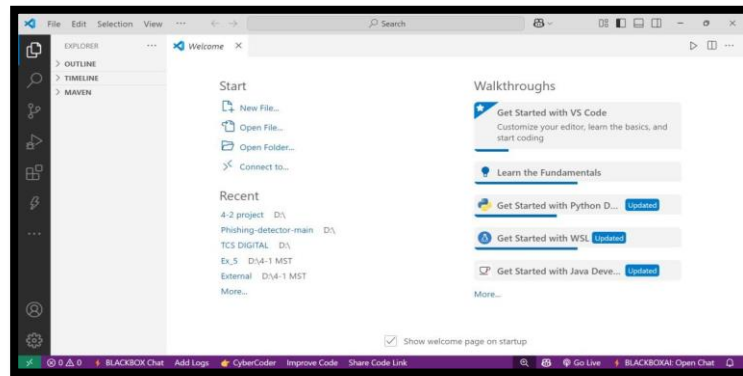
In this project, VS Code was used as the primary development environment for integrating all system components. The frontend of the application was developed using **HTML, CSS, and JavaScript**, providing a user-friendly interface for uploading X-ray images and interacting with the chatbot.

The backend was implemented using **FastAPI**, which handled API requests, model inference, and communication between different modules. The trained deep learning models (ResNet-101, EfficientNet-B0, and U-Net) were downloaded and integrated into the backend for real-time prediction.

Additionally, **Ollama** was integrated within the VS Code environment to run the

LLaMA3 model locally for the RAG-based chatbot. This allowed seamless interaction between the frontend, backend, and AI models, ensuring efficient processing and response generation.

Overall, VS Code enabled smooth integration of frontend technologies, backend services, and AI models into a unified system.



Screen 3: Visual Studio Code Interface

5.5 EXPLANATION OF KEY FUNCTIONS

5.5.1 DEEP LEARNING MODELS USED

A. Convolutional Neural Network (CNN)

Convolutional Neural Networks (CNNs) are widely used deep learning models for image classification tasks. They automatically learn spatial features from input images using convolutional filters, pooling layers, and fully connected layers. CNNs are particularly effective in medical image analysis, where identifying patterns such as textures and abnormalities is essential.

In this project, CNNs are used for detecting multiple lung diseases from chest X-ray images. The input images are resized to a fixed size of **224 × 224 pixels** and normalized before being passed into the network.

The architecture mainly consists of:

- **Convolutional Layers:** Extract important features such as edges and textures
- **Pooling Layers:** Reduce spatial dimensions and computational complexity
- **Fully Connected Layers:** Perform final classification

The structure of CNN includes convolutional layers, pooling layers, and fully connected layers:

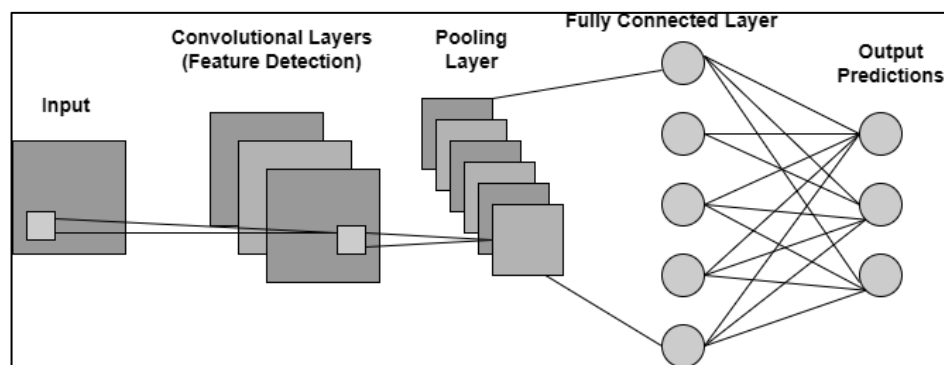


Fig 5.1 Structure of CNN

The structure of CNN is shown in Fig.5.1 It consists of an input layer, multiple hidden layers, and an output layer. The hidden layers typically include convolutional layers, pooling layers, fully connected layers, and sometimes normalization layers. Convolutional layers apply filters to the input, capturing local dependencies and extracting features like edges and textures. Pooling layers reduce the spatial dimensions of the data, lowering the computational load and the model's sensitivity to the location of features in the input. Fully connected layers integrate these features, which are then used for tasks such as classification or regression in the output layer. The activation functions, especially ReLU, introduce non-linearity, enabling the CNN to learn complex patterns. CNNs are renowned for their effectiveness in image and video recognition, image classification, and other visual data driven tasks.

Convolutional Layer: This layer performs feature extraction using a filter matrix over localized regions of the input. The convolution operation can be represented as:

$$A_{xy}^k = \sigma(\sum_p \sum_q W_{pq}^k \cdot I_{(x+p)(y+q)} + B^k) \quad (1)$$

Pooling Layer: This layer downsamples the input to reduce spatial dimensions while preserving the most important features. Max pooling, a commonly used method, is represented as:

$$P_{xy} = \max(R_{ij}) \quad (2)$$

Where, P_{xy} is the pooled value at position (x,y), R_{ij} represents the set of values in the pooling window.

Fully Connected Layer: This layer connects all neurons from the previous layer to each output node. The transformation is defined as:

$$Z_n = \sigma(\sum_m W_{nm} \cdot F_m + B_n) \quad (3)$$

Where, Z_n is the output, W_{nm} is the weight connecting input m to output n, A_m is the flattened feature map and B_n is the bias

B. ResNet-101 Model:

ResNet-101 is a deep convolutional neural network that uses residual connections to overcome the vanishing gradient problem. These skip connections allow the model to learn deeper representations effectively.

In this project:

- ResNet-101 is used for **multi-label disease classification**
- It outputs probability scores for multiple lung diseases
- The model is fine-tuned on the NIH Chest X-ray dataset

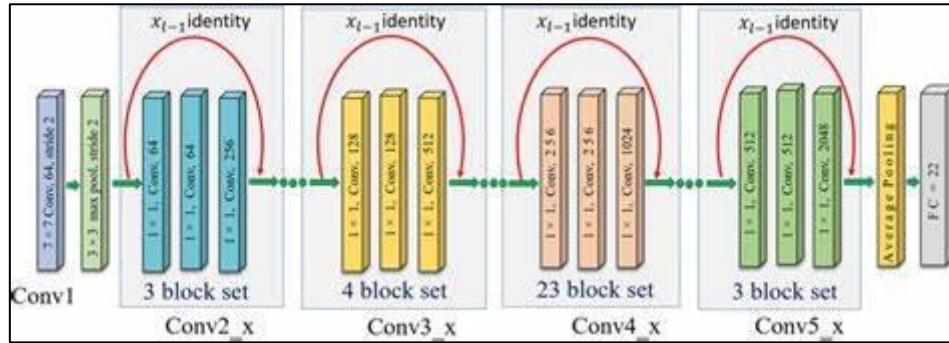


Fig 5.2 : Structure of Resnet-101

C. EfficientNet-B0 Model:

EfficientNet-B0 is a lightweight and efficient CNN architecture that scales depth, width, and resolution in a balanced manner. It achieves high accuracy with fewer parameters compared to traditional CNNs.

In this project:

- EfficientNet-B0 is used alongside ResNet-101
- It provides complementary predictions
- Helps improve overall performance when combined

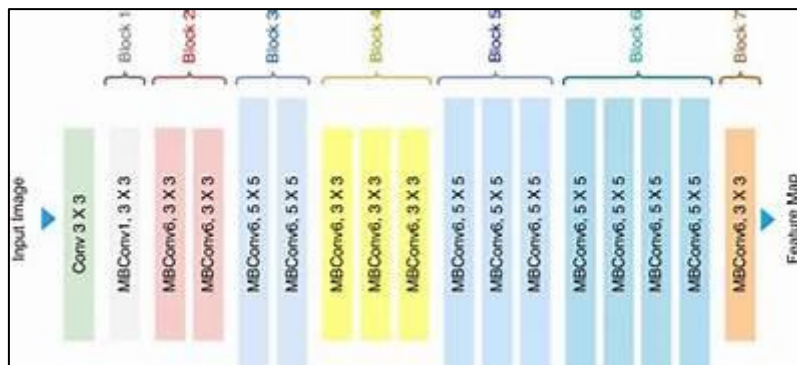


Fig 5.3 :Structure of EfficientNet-B0

D. U-Net for Lung Segmentation:

U-Net is a convolutional neural network designed for biomedical image segmentation. It consists of an encoder-decoder architecture that captures both spatial and contextual information.

In this project:

- U-Net is trained on the **JSRT dataset**
- It generates **lung masks** from X-ray images
- The mask is used to highlight lung regions

E. Ensemble Learning:

To improve prediction accuracy, outputs from ResNet-101 and EfficientNet-B0 are combined using an ensemble approach:

$$\text{Final Prediction} = \frac{P_{ResNet} + P_{EffNet}}{2} \quad (4)$$

This reduces model bias and improves robustness.

F. Threshold-Based Multi-label Classification:

Unlike single-label classification, this project uses multi-label classification, where an image can have multiple diseases.

From your code:

- Each disease has a separate threshold
- Thresholds are optimized using F1-score on validation data
- Predictions are selected only if:

$$P_i \geq T_i \quad (5)$$

Where:

P_i = predicted probability

T_i = optimal threshold for class i

G. Ollama (LLaMA3 Model for RAG Chatbot):

Ollama is a framework used to run large language models locally. In this project, it is used to run the LLaMA3 model as part of the Retrieval-Augmented Generation (RAG) pipeline.

In this system:

- Ollama is used to generate **context-aware medical responses**
- It takes input from the user query along with retrieved medical documents
- Produces meaningful and human-like clinical explanations

5.5.2 EXPLANATION OF THE DATA USED

The dataset used in this project is obtained from publicly available medical image repositories, primarily the NIH Chest X-ray Dataset for disease classification and the JSRT Dataset for lung segmentation.

The NIH Chest X-ray dataset contains a large collection of chest radiographs labeled with multiple lung diseases. It is widely used for multi-label classification tasks in medical imaging. Each image is associated with one or more disease labels, making it suitable for training deep learning models for detecting multiple conditions simultaneously.

The JSRT dataset is used specifically for lung segmentation. It contains annotated chest X-ray images with corresponding lung masks, which are used to train the U-Net model for accurately extracting lung regions.

In addition to image datasets, custom medical text data is used for the RAG-based chatbot. This data consists of curated information related to 14 different lung diseases, including symptoms, causes, diagnosis, and treatment details. These documents are stored as text files and later converted into embeddings for efficient retrieval using a FAISS vector database.

DATASETS USED:

NIH CHEST X-RAYS :

- a. Type of dataset: Image Dataset
- b. No. of training images: ~80,000 images
- c. No. of testing images: ~32,000 images
- d. Dataset source: NIH Chest X-Ray Dataset (Kaggle)
- e. Diseases: Atelectasis, Cardiomegaly, Effusion, Infiltration, Mass, Nodule, Pneumonia, Pneumothorax, Edema, Fibrosis, Pleural Thickening, Emphysema, Hernia, Consolidation

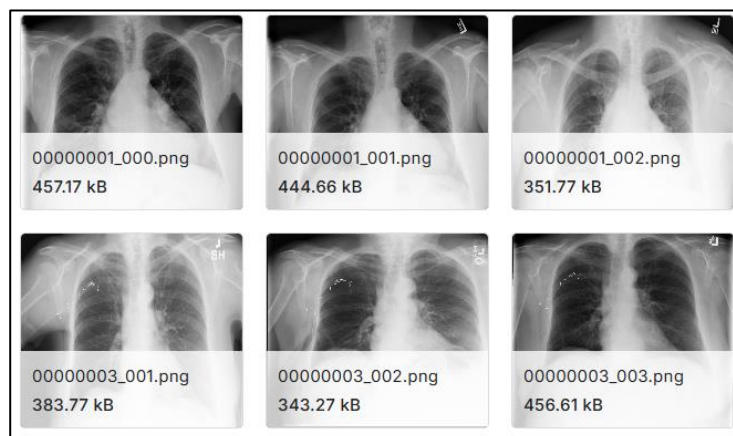


Fig 5.4: Sample dataset of NIH CHEST-XRAY

JSRT DATASET:

- a. Type of dataset: Image Dataset (Chest X-ray Images)
- b. No. of training images: ~200 images
- c. No. of testing images: ~47 images
- d. Dataset source: Japanese Society of Radiological Technology (JSRT Dataset)
- e. Annotations: Lung masks (left and right lung segmentation)
- f. Purpose: Used for training the U-Net model for lung segmentation

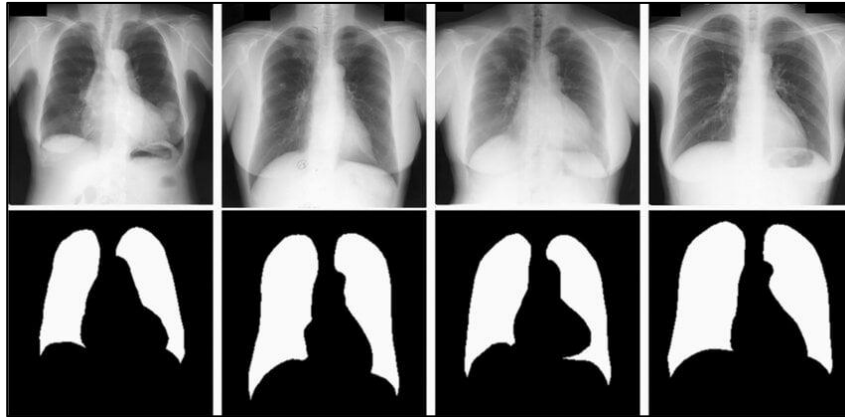


Fig 5.5: Sample dataset of JSRT

RAG CHATBOT DATA:

- a.Type of dataset: Text Dataset (Medical Knowledge Base)
- b. Data format: Structured text files (.txt)
- c. Data sources: World Health Organization (WHO) and Mayo Clinic websites
- d. Content: Disease-related information including symptoms, causes, diagnosis
- e. No. of documents: Multiple disease-specific documents (one file per disease)
- f. Preprocessing: Text splitting into smaller chunks and conversion into embeddings
- g. Storage: Stored in FAISS vector database for efficient retrieval

Data Preprocessing and Extraction Steps:

The collected data undergoes several preprocessing steps before being used in the system:

Image Loading: Chest X-ray images are loaded from dataset directories using their file names.

Resizing: All images are resized to **224 × 224 pixels** to maintain consistency across models.

Colour Conversion: Images are converted from BGR to RGB format for classification and to grayscale for segmentation.

Normalization: Pixel values are scaled using preprocessing functions to improve model performance.

Label Extraction: Disease labels are extracted in a multi-label format using a label mapping structure, where each image corresponds to a vector of disease indicators.

Segmentation Mask Preparation: Lung masks from the JSRT dataset are processed and binarized for training the U-Net model.

Text Data Processing (RAG): Medical text files are cleaned and converted into embeddings using an embedding model. These embeddings are stored in a **FAISS vector database** for fast similarity search.

Features in the Dataset:

1. **Image Data:** Chest X-ray images representing different lung conditions.
2. **Disease Labels:** Multi-label annotations indicating diseases such as pneumonia, fibrosis, and atelectasis.
3. **Segmentation Masks:** Binary masks representing lung regions used for segmentation.
4. **Medical Text Data (RAG):** Custom documents containing information about lung diseases, used by the Ollama-based LLaMA3 model to generate responses.

5.5.3 TRAINING THE MODEL

The models are trained using large-scale medical datasets with proper preprocessing and validation strategies.

Training Process:

- Input images are resized to **224 × 224**
- Pixel values are normalized using preprocessing functions
- Data is split into:
 - Training set
 - Validation set
 - Testing set
- CNN models (ResNet-101 & EfficientNet-B0) are trained on the **NIH dataset**
- U-Net is trained separately on the **JSRT dataset**

During training:

- Model predictions are compared with true labels
- Loss is minimized using optimization algorithms
- Best models are saved based on validation performance

Threshold Optimization

Instead of using a fixed threshold, the system computes **optimal thresholds for each class**:

- Predictions are generated on validation set
- Threshold values from **0.1 to 0.9** are tested
- Best threshold is selected based on **maximum F1-score**

This improves performance for imbalanced datasets.

HYPERPARAMETER TUNING

Hyperparameters play an important role in model performance.

Image Size: All images are resized to **224 × 224** for consistency.

Threshold Range: Thresholds are searched between **0.1 and 0.9** to find optimal values.

Batch Processing: Data is processed in batches to improve training efficiency.

Optimizer: Optimization algorithms are used to minimize loss and improve accuracy.

Activation Function: Sigmoid activation is used for multi-label classification, producing outputs between 0 and 1.

5.6 METHOD OF IMPLEMENTATION

5.6.1 SAMPLE CODE

A. FRONT END

The frontend of the PulmoAI system is designed using HTML, CSS, and JavaScript, providing a visually rich, interactive, and user-friendly interface for lung disease detection and medical question answering. The interface is integrated with the backend APIs to enable real-time predictions and chatbot responses.

The system contains of 3 main frontend pages:

- Landing page
- Analysis dashboard
- Authentication page

1. Landing Page (Home Interface)

The landing page provides an overview of the system, highlighting its capabilities such as lung disease detection, segmentation, and AI-powered medical assistance.

It is designed with:

- Modern UI using gradient themes and soft animations
- Navigation bar with login/signup options
- System highlights and model descriptions
- Call-to-action buttons for accessing the application

NAV BAR

```
<nav>
  <div class="nav-logo">PulmoAI</div>
  <div class="nav-links">
    <a href="#">Home</a>
    <a href="#">Models</a>
  </div>
</nav>
```

HEADER

```
<div class="hero">
  <h1>PulmoAI</h1>
  <p>AI-powered pulmonary diagnostics
system</p>
  <button>Start Analysis</button>
</div>
```

D

2. Authentication Interface (Login & Signup)

The authentication page allows users to securely access the system.

Features:

- Login and Registration tabs
- Email and password input fields
- Password validation and strength indicator
- Session-based authentication

Key Functionalities:

- User login validation
- New user registration
- Error handling and notifications
- Secure input handling

LOGIN FORM

```
<input type="email" placeholder="Email"/>
<input type="password"
placeholder="Password"/>
<button>Login</button>
```

JS INTERACTION AND VALIDATION

```
function switchAuth(type) {
  document.getElementById("login-panel").style.display = "none";
  document.getElementById("signup-panel").style.display = "block";
}
if(password.length < 6){
  alert("Weak password");
}
```

3. Analysis Dashboard Interface

The main working interface of the system is the Analysis Dashboard, where users interact with AI models.

It contains two major modules:

A. Lung Disease Prediction Module

This module allows users to upload chest X-ray images and obtain predictions.

Functional Flow:

- Upload image

- Preprocess and send to backend
- Display prediction results

Features:

- Drag-and-drop image upload
- Image preview before analysis
- Analyze button to trigger prediction
- Visualization panels for:
 - Original image
 - Segmentation output
 - Grad-CAM heatmap

Results Display:

- Predicted diseases
- Confidence scores
- Probability bars
- Detection status

ANALYSIS CODE:

```
<input type="file" id="upload"/>
<button onclick="analyze()">Analyze</button>
function analyze(){
  let file = document.getElementById("upload").files[0];
  // send to backend
}
<div class="results">
  <p>Disease: Pneumonia</p>
  <p>Confidence: 92%</p>
</div>
```

B. RAG-Based Medical Chatbot

This module enables users to ask medical-related questions.

Functional Flow:

- User enters query
- Query sent to backend
- Response generated using RAG pipeline
- Answer displayed with sources

Features:

- Text input for medical questions
- Quick suggestion buttons

- Context-aware responses
- Display of sources (retrieved documents)

RAG CODE:

```
<textarea id="question"></textarea>
<button onclick="ask()">Ask</button>
function ask(){
  let q = document.getElementById("question").value;
  // send to backend
}
fetch("/predict", {
  method: "POST",
  body: formData
})
.then(res => res.json())
.then(data => {
  console.log(data);
});
```

4. JavaScript Integration

JavaScript is used to connect frontend with backend APIs.

Main Functions:

- Sending image data to prediction API
- Fetching prediction results
- Sending user queries to chatbot
- Displaying responses dynamically

Example Workflow:

- Capture user input
- Send request using fetch API
- Receive JSON response
- Update UI dynamically

5. UI/UX Design Features

The frontend is designed to ensure:

- Smooth navigation between modules
- Responsive and interactive UI
- Real-time feedback to user actions
- Clear visualization of results

Key Design Elements:

- Gradient-based modern theme
- Animated transitions

B. BACK END

The backend of the PulmoAI system is implemented using **FastAPI in Visual Studio Code**, where trained models from Kaggle are integrated for real-time prediction and chatbot responses.

MODEL TRAINING:

UNET is trained for 25 epochs .RESNET-101 is trained for 15 epochs ,Efficient Net B0 is also trained for 15 epochs all the training happened in Kaggle notebook .

```
seg_model = build_unet()
seg_model.compile(optimizer=Adam(1e-4), loss="binary_crossentropy", metrics=["accuracy"])
seg_model.fit(seg_train_ds, validation_data=seg_val_ds, epochs=25)
seg_model.evaluate(seg_test_ds)
seg_model.save("UNET_lung.h5")
print(" UNET_lung.h5 saved")
# — RESNET-101

callbacks = [
    tf.keras.callbacks.ModelCheckpoint(
        "best_resnet.h5", monitor="val_auc", mode="max",
        save_best_only=True, save_weights_only=False), # ← full model
    tf.keras.callbacks.EarlyStopping(
        monitor="val_auc", mode="max", patience=4, restore_best_weights=True),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor="val_auc", mode="max", factor=0.3, patience=2)
]
res_base = ResNet101(include_top=False, weights="imagenet", input_shape=(224,224,3))
for l in res_base.layers[:-40]: l.trainable = False
for l in res_base.layers[-40:]: l.trainable = True
resnet_model = build_model(res_base)
resnet_model.compile(optimizer=Adam(1e-4), loss=weighted_bce(pos_weight), metrics=metrics)
print("\nTraining ResNet-101...")
resnet_model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS, callbacks=callbacks)
print(" best_resnet.h5 saved")
# — EFFICIENTNET-B0

effnet_callbacks = [
    tf.keras.callbacks.ModelCheckpoint(
        "best_effnet.h5", monitor="val_auc", mode="max",
        save_best_only=True, save_weights_only=False), # ← full model
    tf.keras.callbacks.EarlyStopping(
        monitor="val_auc", mode="max", patience=4, restore_best_weights=True),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor="val_auc", mode="max", factor=0.3, patience=2)
]
eff_base = EfficientNetB0(include_top=False, weights="imagenet", input_shape=(224,224,3))
for l in eff_base.layers: l.trainable = False
effnet_model = build_model(eff_base)
effnet_model.compile(optimizer=Adam(1e-4), loss=weighted_bce(pos_weight), metrics=metrics)
print("\nTraining EfficientNet-B0...")
effnet_model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS, callbacks=effnet_callbacks)
effnet_model.load_weights("best_effnet.h5")
print(" best_effnet.h5 saved")
```

BCE WEIGHTED LOSS:

This loss used in training resnet-101 and efficientNetB0. This loss function is used to handle **class imbalance** in the dataset, where some diseases occur less frequently than others. It assigns higher weight to rare classes, ensuring the model learns all diseases effectively and improves prediction accuracy.

```
pos_counts = label_df.sum().values
neg_counts = len(label_df) - pos_counts
pos_weight = tf.constant(neg_counts / (pos_counts + 1e-6), dtype=tf.float32)

def weighted_bce(pos_weight):
    def loss(y_true, y_pred):
        y_pred = tf.clip_by_value(y_pred, 1e-7, 1-1e-7)
        return tf.reduce_mean(-pos_weight*y_true*tf.math.log(y_pred) - (1-
y_true)*tf.math.log(1-y_pred))
    return loss
```

MODEL LOADING IN VSCODE AND PREDICTION :

After training models were loaded to vscode to predict new input

```
app = FastAPI(title="PulmoAI")
seg_model = tf.keras.models.load_model("unet_lung.h5")
resnet_model = tf.keras.models.load_model("best_resnet.h5")
effnet_model = tf.keras.models.load_model("best_effnet.h5")
@app.post("/classify")
async def classify(file: UploadFile):

    img = cv2.imdecode(...)
    preds = resnet_model.predict(img)

    return {"predictions": preds}
probs = ALPHA * p_r + (1 - ALPHA) * p_e
det = [d for d, p in preds.items() if p >= threshold]
cam = _gradcam(model, input, class_idx, image)
```


RAG-CHATBOT:

```
llm = Ollama(model="llama3")
result = qa.invoke({"query": question})
db = FAISS.load_local(INDEX_PATH,
embeddings)
retriever = db.as_retriever()
```

5.6.2 RESULTS AND ANALYSIS

The results of the proposed PulmoAI system demonstrate its effectiveness in accurately detecting multiple lung diseases from chest X-ray images. The performance was evaluated using metrics such as AUC, accuracy, and F1-score across ResNet-101, EfficientNet-B0, and their ensemble (fusion) model. Among these, the fusion model achieved the highest overall performance, with a mean AUC of 0.7699, showing improved generalization compared to individual models. ResNet-101 provided strong baseline performance, while EfficientNet-B0 contributed to enhancing feature diversity in the ensemble. Disease-wise analysis indicates that the model performs particularly well in detecting conditions such as Hernia, Emphysema, and Cardiomegaly, while maintaining stable performance across other classes. Training and validation curves reveal that although minor overfitting is observed, the ensemble approach helps in stabilizing predictions and reducing performance gaps. Overall, the results confirm that combining multiple deep learning models significantly improves prediction accuracy, robustness, and reliability, making the system suitable for real-world medical diagnosis support.

ResNet-101 Training Analysis

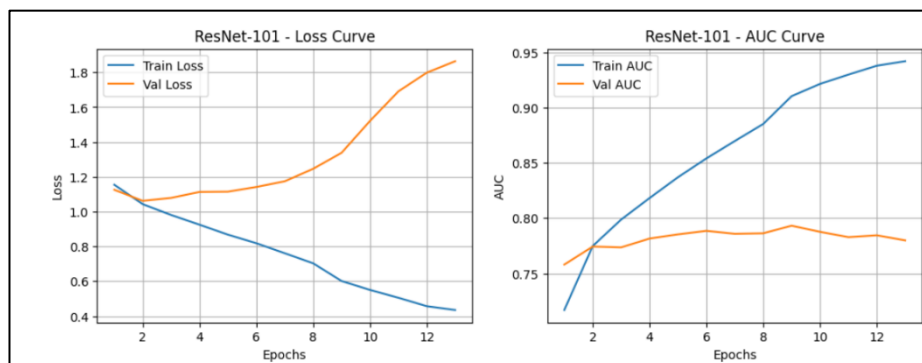


Fig 5.6 :Traning and validation curves of RESNET-101

The training and validation curves of the ResNet-101 model provide insights into its learning behaviour. From the loss curve, it can be observed that the training loss decreases steadily with

increasing epochs, indicating that the model is effectively learning patterns from the dataset. However, the validation loss initially decreases but starts increasing after a few epochs, suggesting the presence of overfitting.

Similarly, the AUC curve shows a consistent improvement in training AUC, reaching a high value close to 0.94, which reflects strong learning capability. The validation AUC improves in the initial epochs and stabilizes around 0.78–0.79, with the best validation AUC of approximately 0.7932 achieved at epoch 9. This indicates that the model performs well but begins to overfit after this point, making early stopping an effective strategy.

EfficientNet-B0 Training Analysis:

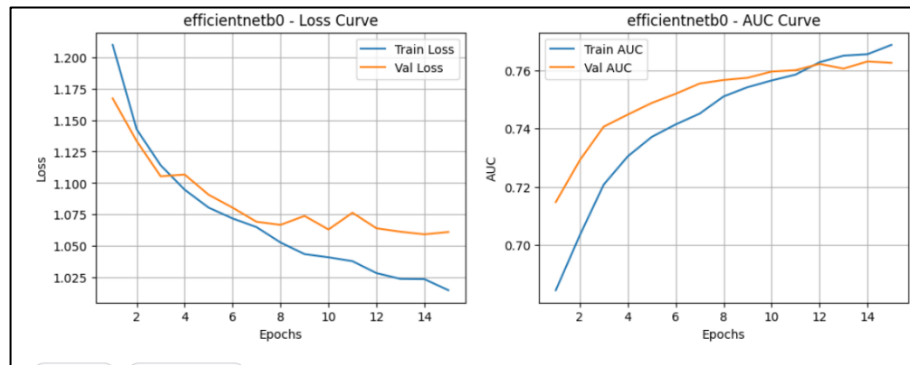


Fig 5.7: Training and validation curves of EFFICIENT NETB0

The EfficientNet-B0 model demonstrates a more stable learning pattern compared to ResNet-101. The loss curve shows a gradual decrease in both training and validation loss, with only minor fluctuations, indicating better generalization and reduced overfitting.

In the AUC curve, both training and validation AUC increase steadily, with the validation AUC closely following the training curve. The model achieves a best validation AUC of approximately 0.7630 at epoch 14, showing consistent performance across epochs. Unlike ResNet, EfficientNet does not exhibit a significant gap between training and validation metrics, highlighting its robustness and efficiency.

Model Performance Based on AUC

Table 4: mean AUC

	Model	Mean AUC
0	ResNet	0.765091
1	EfficientNet	0.723600
2	Fusion	0.769865

The Mean AUC (Area Under Curve) values provide a measure of the model's ability to distinguish between different lung diseases. From the results, the ResNet model achieves a Mean AUC of 0.7651, while EfficientNet-B0 achieves 0.7236, indicating slightly lower discriminative performance.

The ensemble (fusion) model, which combines predictions from both ResNet and EfficientNet, achieves the highest Mean AUC of 0.7699. This improvement demonstrates that combining multiple models helps in capturing complementary features and enhances overall classification performance.

Model Performance Based on Accuracy and F1 Score:

Table 5: AUCCARCY AND F1 SCORE

	Model	Accuracy	F1 Score
0	ResNet	0.852538	0.277677
1	EfficientNet	0.797806	0.217255
2	Fusion	0.837717	0.284215

The evaluation based on Accuracy and F1 Score further highlights the effectiveness of the models. The ResNet model achieves an accuracy of 85.25%, while EfficientNet-B0 achieves 79.78%, indicating that ResNet performs better as a standalone model.

However, the fusion model achieves an accuracy of 83.77%, which is slightly lower than ResNet but still competitive. Importantly, the fusion model records the highest F1 Score of 0.2842, compared to ResNet (0.2777) and EfficientNet (0.2173).

Since F1 Score balances precision and recall, this result indicates that the ensemble model provides better overall classification balance, especially for multi-label disease prediction.

The evaluation based on Accuracy and F1 Score further highlights the effectiveness of the models. The ResNet model achieves an accuracy of 85.25%, while EfficientNet-B0 achieves 79.78%, indicating that ResNet performs better as a standalone model.

However, the fusion model achieves an accuracy of 83.77%, which is slightly lower than ResNet but still competitive. Importantly, the fusion model records the highest F1 Score of 0.2842, compared to ResNet (0.2777) and EfficientNet (0.2173).

Since F1 Score balances precision and recall, this result indicates that the ensemble model provides better overall classification balance, especially for multi-label disease prediction.

Disease-wise Performance Analysis:

Table 6: AUC of each disease

	Disease	ResNet AUC	EfficientNet AUC	Fusion AUC
0	Atelectasis	0.725118	0.677058	0.720746
1	Cardiomegaly	0.844105	0.761656	0.841852
2	Consolidation	0.714326	0.682492	0.715488
3	Edema	0.820180	0.805588	0.828135
4	Effusion	0.790608	0.738657	0.789612
5	Emphysema	0.860706	0.744821	0.844750
6	Fibrosis	0.746896	0.769009	0.780910
7	Hernia	0.879140	0.874606	0.901394
8	Infiltration	0.677456	0.656932	0.684181
9	Mass	0.734382	0.652900	0.730678
10	Nodule	0.691734	0.647777	0.696215
11	Pleural_Thickening	0.723620	0.691694	0.729247
12	Pneumonia	0.669469	0.651360	0.679726
13	Pneumothorax	0.833533	0.775845	0.835176

Metrics formulae

Mean Accuracy

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \quad (6)$$

Precision

$$\text{Precision} = \frac{TP}{TP+FP} \quad (7)$$

Recall

$$\text{Recall} = \frac{TP}{TP+FN} \quad (8)$$

F1 Score

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (9)$$

Mean F1 Score (for multi-label)

$$\text{Mean F1} = \frac{1}{N} \sum_{i=1}^N F1_i \quad (10)$$

TP: True Positive

TN: True Negative

FP: False Positive

FN: False Negative

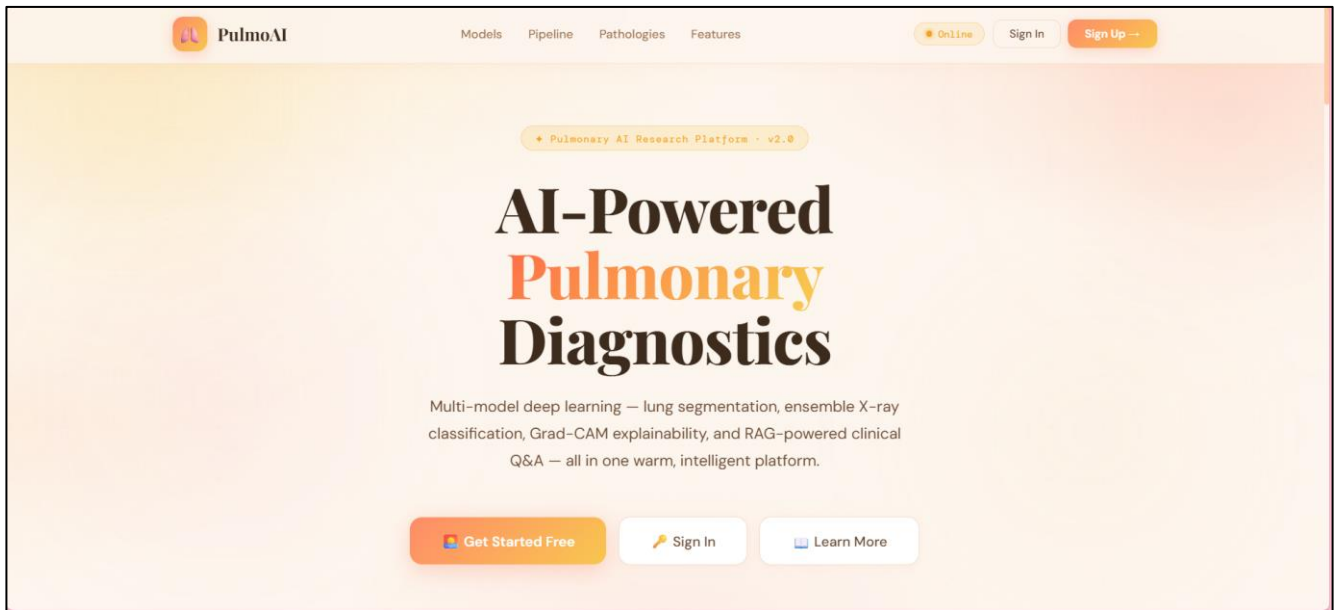
The disease-wise AUC values provide a detailed evaluation of how well each model performs across different lung diseases. It can be observed that the performance varies depending on the complexity and visual characteristics of each disease.

The ResNet model shows strong performance across most diseases, achieving high AUC values for conditions such as Hernia (0.8791), Emphysema (0.8607), and Pneumothorax (0.8335). EfficientNet-B0, while slightly lower in most cases, still demonstrates competitive performance and generalization ability.

The fusion (ensemble) model consistently improves or maintains performance across nearly all diseases. Notably, it achieves the highest AUC for Hernia (0.9014), indicating excellent discriminative capability. Similarly, improvements can be seen in diseases like Edema (0.8281), Fibrosis (0.7809), and Pleural Thickening (0.7292).

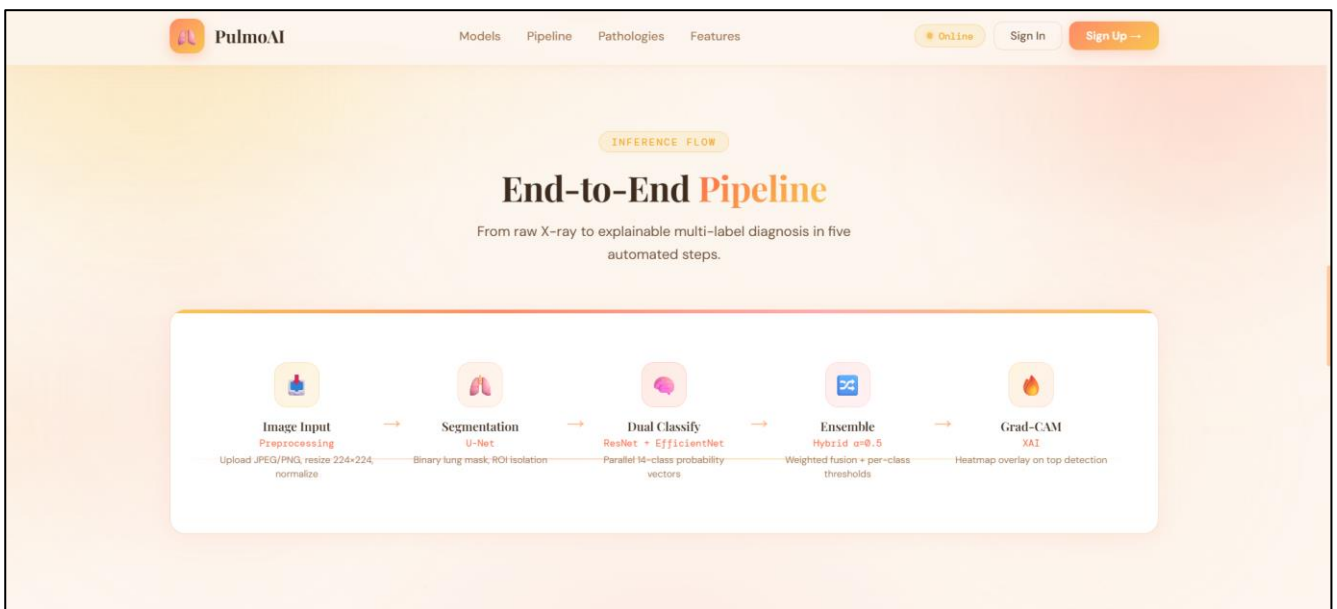
For more challenging classes such as Infiltration, Mass, and Nodule, all models show comparatively lower AUC values, suggesting that these conditions are harder to distinguish due to overlapping features in X-ray images. However, even in these cases, the ensemble model provides slight improvements over individual models.

5.6.3 OUTPUT SCREENS



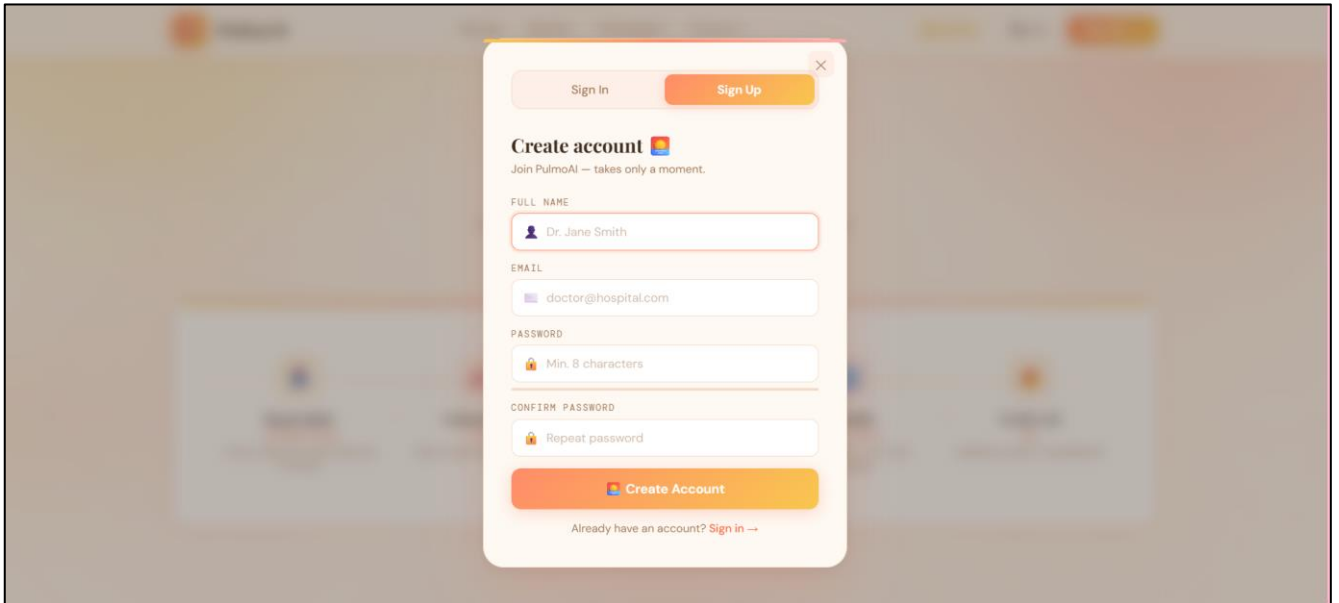
Screen 4 : Home Page

Screen 4 displays The home page presents an AI-powered pulmonary diagnosis platform that integrates deep learning models for lung disease detection, segmentation, and explainability. It offers an intuitive interface for users to upload X-ray images, obtain predictions, and access RAG-based medical insights in a unified system.



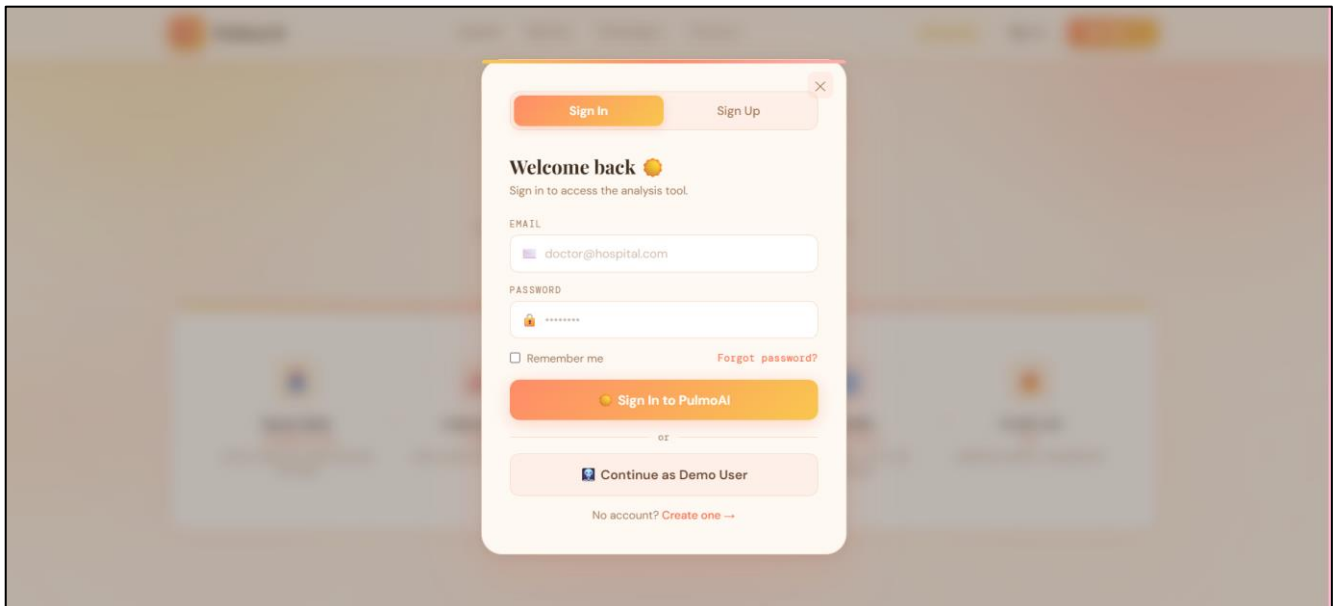
Screen 5: Pipeline of chest x-ray analysis

Screen 5 displays The pipeline illustrates the complete workflow from raw chest X-ray input to final diagnosis using preprocessing, segmentation, classification, and ensemble techniques. It also incorporates Grad-CAM to provide explainable visual insights, making the predictions more interpretable and reliable.



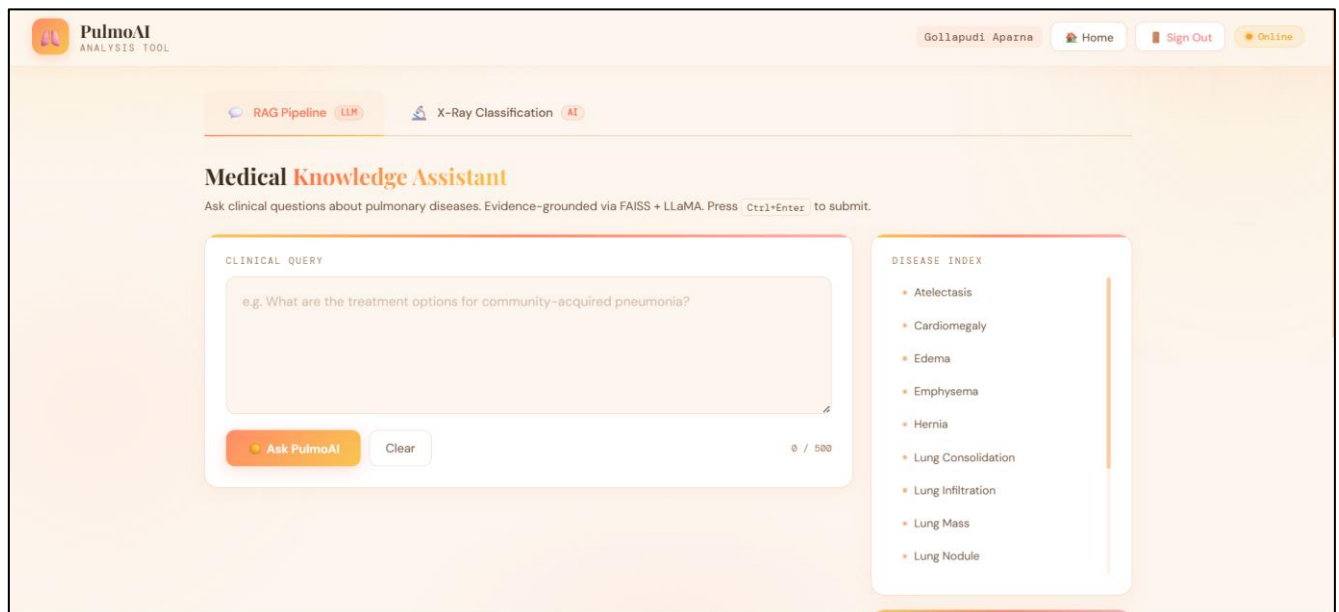
Screen 6: Sign up Page

Screen 6 is the signup Page where new users can create an account by providing their fullname, email, and password. Upon successful registration, users are redirected to the sign page.



Screen 7: Login Page

Screen 7 is the Signin Page that allows existing users to access the system. Users are required to enter valid email and password credentials to proceed.

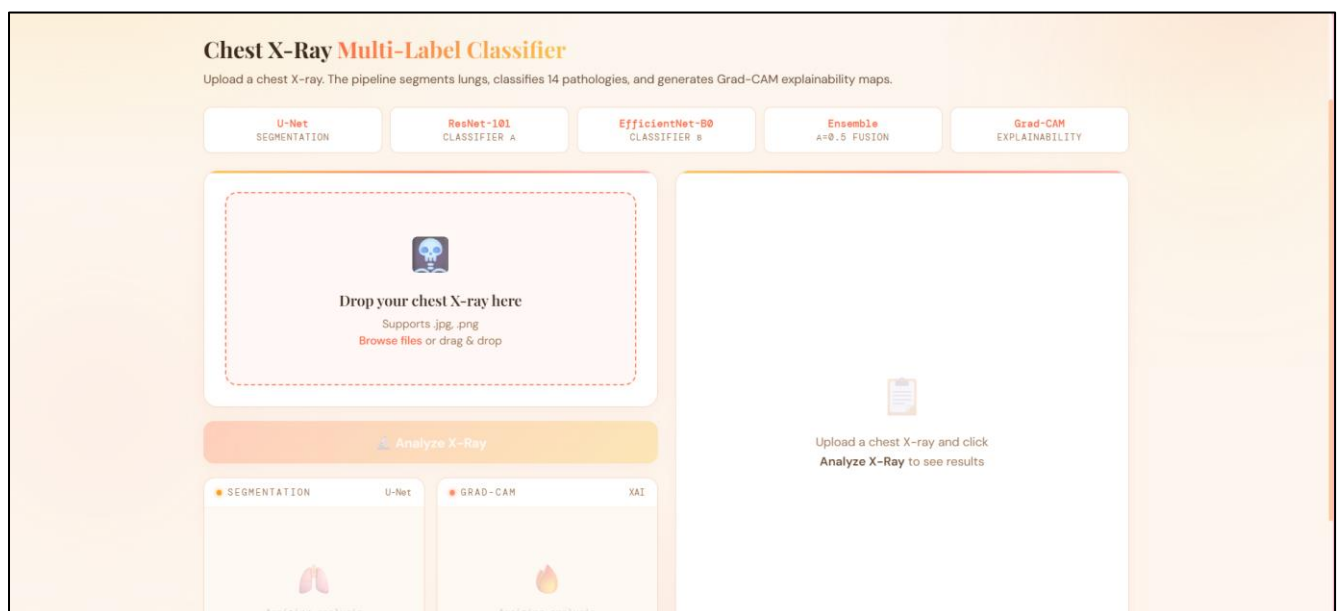


Screen 8: RAG-CHATBOT Dashboard

Screen 8 This interface provides a RAG-based medical knowledge assistant where users can ask clinical questions related to lung diseases. It retrieves relevant information from a FAISS database and generates accurate, context-aware answers using the LLaMA model.

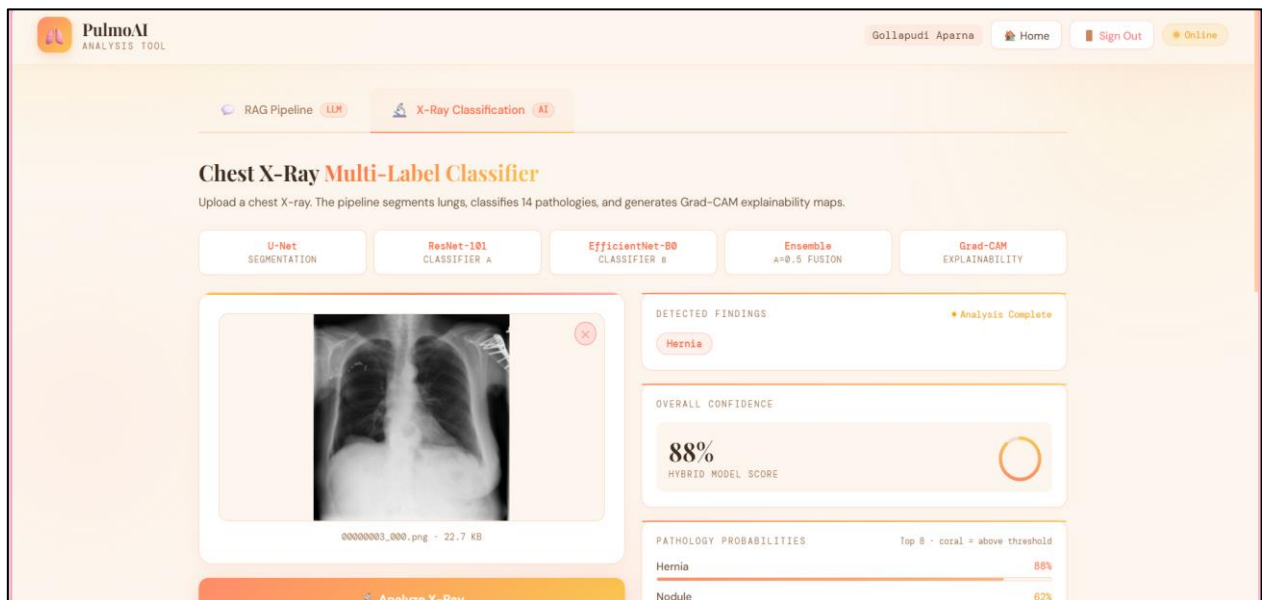
Home: redirects to home page.

X-ray pipeline: redirects to x-ray analysis page.



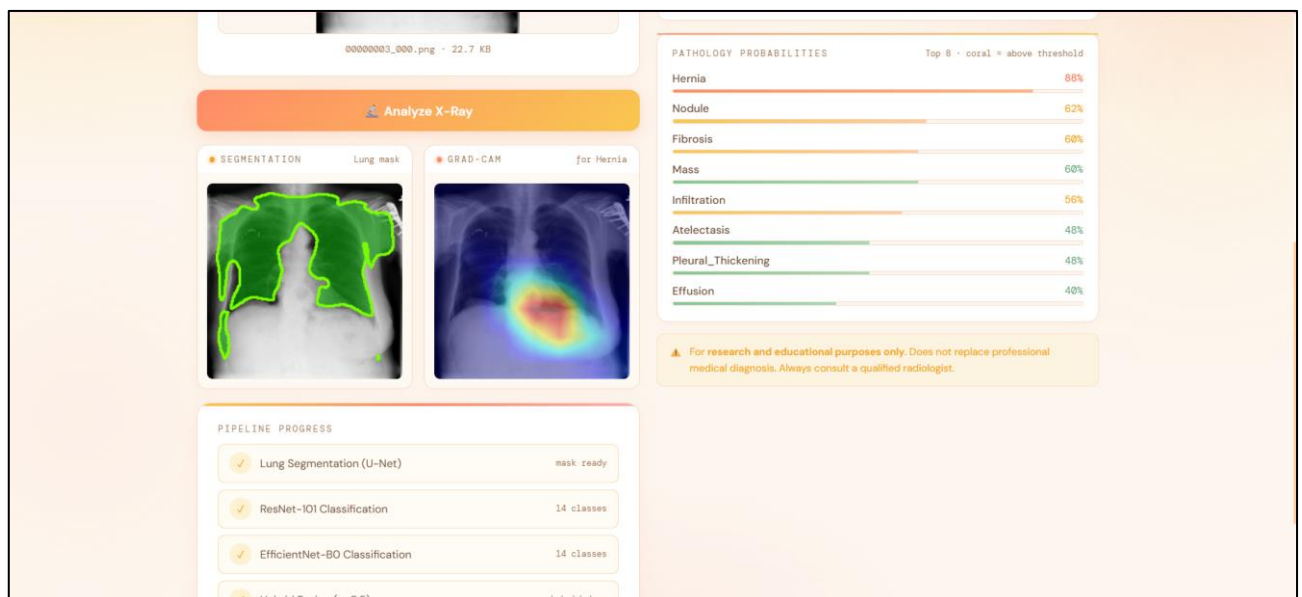
Screen 9: X-RAY ANALYSIS DASHBOARD

Screen 9 displays the interface enables users to upload chest X-ray images and perform multi-label classification for 14 lung diseases using deep learning models. It integrates segmentation, ensemble prediction, and Grad-CAM to provide accurate and explainable diagnostic results.



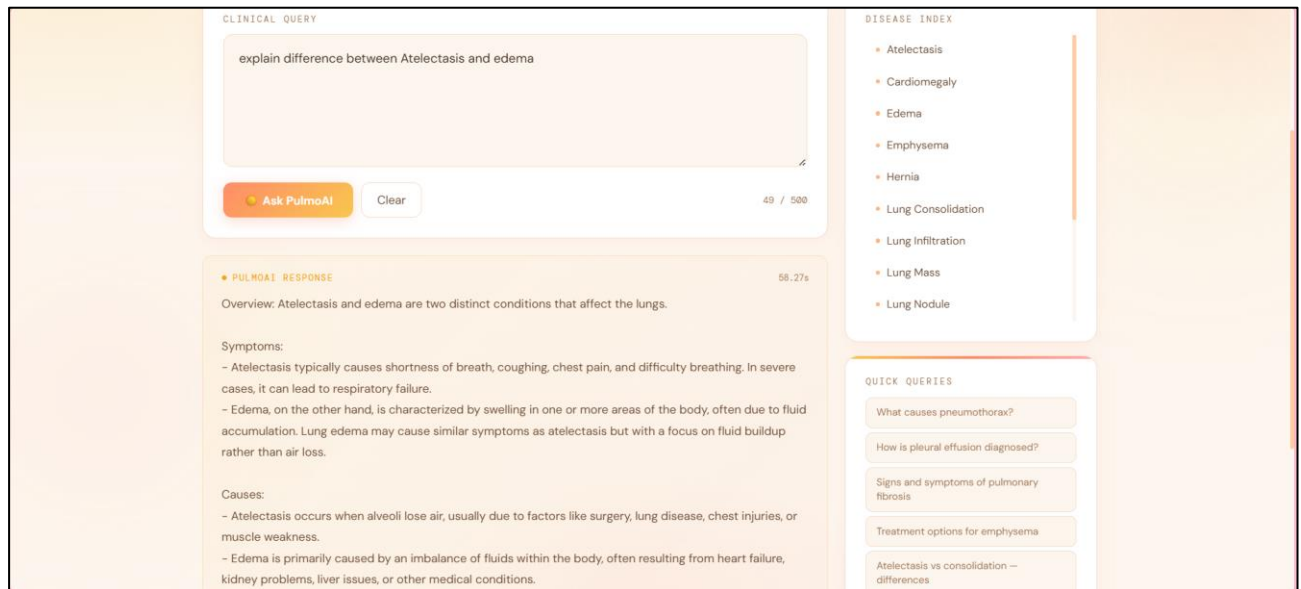
Screen 10: Result Page

Screen 10 This screen displays the analysis results of the uploaded chest X-ray, showing detected diseases along with confidence scores. It also provides detailed pathology probabilities and visual insights to support accurate and explainable diagnosis.



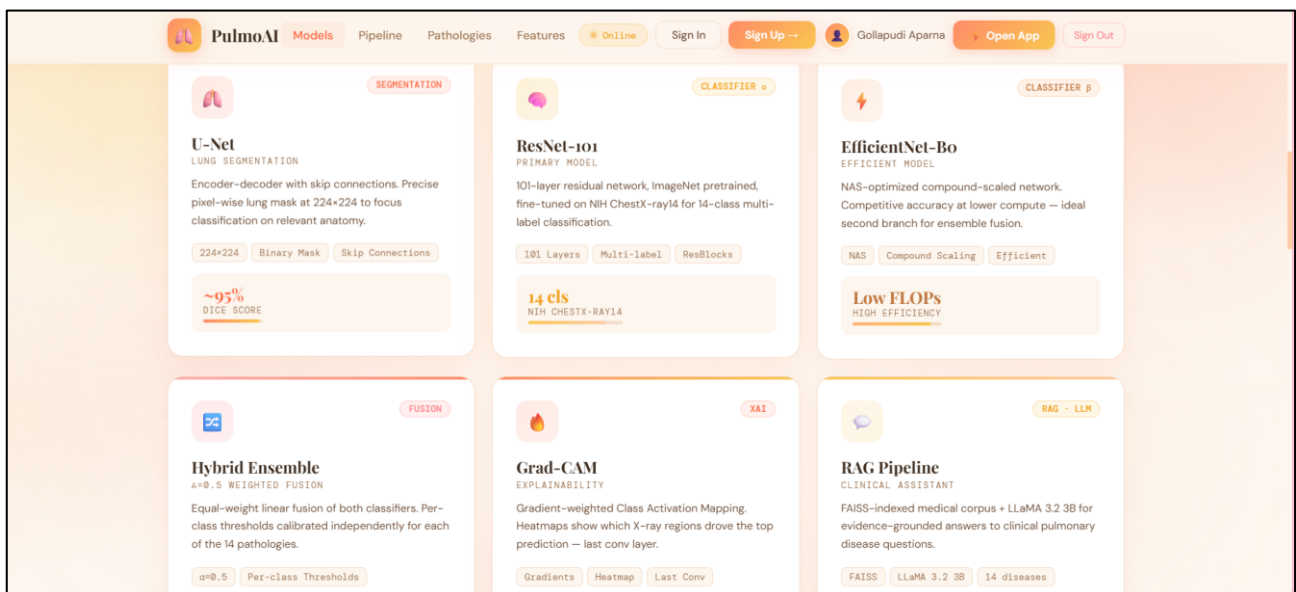
Screen 11: Visualizations

Screen 11 displays the —visualization of the model outputs, including lung segmentation masks and Grad-CAM heatmaps that highlight affected regions in the X-ray. It also displays disease-wise probability scores, helping users understand the model's confidence and decision-making process.



Screen 12: Rag-chatbot prediction

Screen 12 displays the AI-generated clinical response based on the user's medical query using the RAG pipeline. It provides structured explanations, including symptoms and causes, along with relevant medical knowledge for better understanding.



Screen 13: About The Model

Screen 13 showcases the core models used in the system, including U-Net for segmentation, ResNet-101 and EfficientNet-B0 for classification, and Grad-CAM for explainability. It also highlights the hybrid ensemble approach and RAG pipeline.

5.4 CONCLUSION

The **Implementation and Results phase** successfully demonstrated the effectiveness of the proposed PulmoAI system in detecting and analyzing lung diseases from chest X-ray images. The system integrates multiple deep learning models, including ResNet-101 and EfficientNet-B0 for classification, U-Net for lung segmentation, and Grad-CAM for explainability, along with an ensemble approach to improve prediction accuracy. During training, techniques such as weighted loss functions and proper validation strategies were used to handle class imbalance and enhance model performance. The results showed strong AUC, accuracy, and F1-score values, indicating the model's ability to generalize well across multiple diseases. Additionally, the integration of a RAG-based chatbot using FAISS and LLaMA enabled the system to provide context-aware medical insights. Overall, this phase validates the system's capability to deliver accurate, explainable, and clinically relevant predictions, making it suitable for real-world healthcare applications.

6. TESTING AND VALIDATION

6.4 INTRODUCTION

Testing and validation are essential stages in the development of the proposed system, as they help in evaluating the performance, reliability, and accuracy of the model. In this project, testing is carried out on a lung disease detection system that analyzes chest X-ray images using deep learning techniques.

The primary dataset used for testing is the NIH Chest X-ray dataset, which contains a large collection of labeled images representing 14 different lung diseases. A predefined test dataset is used to evaluate the trained models on unseen data, ensuring that the system generalizes well and performs effectively in real-world scenarios.

In addition to the NIH dataset, the JSRT (Japanese Society of Radiological Technology) dataset is utilized during the training phase to enhance image preprocessing. A U-Net model is trained on the JSRT dataset for lung segmentation, enabling precise extraction of the lung regions from chest X-ray images. This segmentation model is then applied to the NIH dataset as a preprocessing step, improving the quality of input data by removing irrelevant regions and focusing only on the lungs. This significantly enhances the feature learning capability of the classification models.

The proposed system utilizes two deep learning models, namely ResNet and EfficientNet, for feature extraction and classification of chest X-ray images. Furthermore, a fusion approach is implemented by combining the predictions of both models using weighted averaging. This ensemble technique improves overall performance and reduces prediction errors.

During the testing phase, each input image undergoes preprocessing steps such as resizing, normalization, and format conversion. Additionally, lung segmentation using the trained U-Net model is applied to ensure that only the relevant regions are analyzed. The processed images are then fed into the classification models. The models generate probability scores for each disease, which are converted into binary outputs using predefined threshold values specific to each disease.

The performance of the system is evaluated using standard metrics such as Accuracy, F1-score, and Area Under Curve (AUC). These metrics provide a comprehensive understanding of the model's effectiveness in multi-label classification and its ability to distinguish between different disease conditions.

Thus, the testing and validation phase ensures that the proposed system is accurate, reliable, and capable of assisting in the detection of lung diseases from chest X-ray images, with improved performance due to the integration of U-Net-based lung segmentation using the JSRT

6.4.1 Scope

The scope of testing and validation in this project is to evaluate the performance and reliability of the lung disease detection system using chest X-ray images. It focuses on verifying whether the proposed deep learning models are capable of accurately identifying multiple lung diseases from unseen data.

The testing process covers the complete pipeline, including dataset loading, image preprocessing, lung segmentation, model inference, threshold-based classification, and performance evaluation. The system is primarily tested on the NIH Chest X-ray dataset, which consists of labeled images corresponding to 14 different lung diseases, ensuring a comprehensive evaluation across multiple classes.

In addition, the scope includes the use of the JSRT dataset for training a U-Net model specifically designed for lung segmentation. The trained U-Net model is applied during preprocessing to extract the lung regions from chest X-ray images. This step enhances the quality of input data by removing irrelevant background information and improving the focus of the classification models on critical regions.

The scope also includes the assessment of individual models such as ResNet and EfficientNet, as well as the fusion model that combines predictions from both networks. By comparing these models, the effectiveness of the ensemble approach in improving prediction accuracy and robustness is analyzed.

Furthermore, the validation process involves the use of performance metrics such as Accuracy, F1-score, and AUC to measure the efficiency of the system. These metrics help in understanding the model's capability to handle multi-label classification and its ability to distinguish between different disease categories.

However, the scope of this testing is limited to offline evaluation using the available datasets (NIH and JSRT) and does not include real-time clinical deployment or integration with hospital systems. Despite this limitation, the testing process provides a strong indication of the system's practical applicability in medical image analysis.

6.4.2 Future Scope of Research

The proposed lung disease detection system provides a strong foundation for automated medical image analysis; however, several research directions can be explored to further advance its effectiveness and applicability.

One significant area for future research is the development of more advanced deep learning architectures. Exploring transformer-based models, hybrid CNN-transformer frameworks, or newer architectures such as EfficientNet and ResNet variants can improve feature representation and classification performance, especially for complex and overlapping lung disease patterns.

Another important research direction involves multi-modal learning. Instead of relying solely on chest X-ray images, future systems can integrate additional clinical data such as patient history, symptoms, laboratory reports, or CT scan images. Combining multiple data sources can lead to more accurate and clinically relevant predictions.

Improving dataset quality and diversity is also a crucial research opportunity. Future work can focus on collecting larger, more balanced, and diverse datasets that include different demographics, imaging conditions, and rare diseases. This will help improve model generalization and reduce bias.

Further research can be conducted in the area of segmentation and localization. Enhancing lung segmentation models or integrating region-based learning approaches can enable the system to not only classify diseases but also highlight affected regions more precisely, aiding in better diagnosis.

Explainable Artificial Intelligence (XAI) is another promising research area. Developing methods to interpret model predictions using techniques like Grad-CAM, attention maps, or saliency maps can increase transparency and trust, which is critical in healthcare applications.

In addition, future research can explore real-time and edge deployment of the system. Optimizing models for low-resource environments, such as mobile or embedded devices, can make the system accessible in rural or resource-constrained healthcare settings.

Another emerging direction is personalized and adaptive learning systems. Models can be designed to continuously learn from new data and adapt to different hospital environments or patient populations, improving performance over time.

Finally, rigorous clinical validation and collaboration with medical professionals will be essential. Conducting real-world testing, clinical trials, and comparative studies with radiologists can help assess the practical utility and reliability of the system in healthcare environments.

6.4.3 Compatibility

Compatibility refers to the ability of the proposed system to function effectively across different hardware and software environments without any performance issues. In this project, the lung disease detection system is designed to be compatible with standard computing platforms and widely used software tools.

The system is implemented using Python and deep learning frameworks such as TensorFlow and Keras, which are supported on multiple operating systems including Windows, Linux, and macOS. This ensures that the model can be executed in different environments without requiring major modifications.

The application is tested on systems with GPU support to accelerate model inference. At the same time, the system is also capable of running on CPU-based environments, although with reduced performance. This flexibility ensures that the system can be used even in resource-limited setups.

Compatibility is also maintained with respect to input data formats. The system supports standard image formats such as JPEG and PNG, which are commonly used for chest X-ray images. Proper preprocessing techniques ensure that images from different sources can be handled efficiently.

Furthermore, the system is compatible with common data handling and visualization libraries, allowing seamless integration with tools used for analysis and reporting. The use of widely adopted libraries enhances portability and ease of deployment.

Overall, the proposed system demonstrates good compatibility across different platforms, making it adaptable and suitable for practical implementation in various environments.

6.4.4 Input Combinations and Preconditions

Input combinations and preconditions define the necessary requirements and conditions that must be satisfied before executing the testing process of the proposed system. These ensure that the model functions correctly and produces reliable results.

The primary input to the system is chest X-ray images obtained from the NIH dataset. These images may vary in terms of resolution, intensity, and quality. Therefore, all input images must undergo preprocessing steps such as resizing to 224×224 pixels, normalization, and color space conversion before being fed into the model. This ensures consistency in input data and improves model performance.

Another important input is the corresponding label information, which is obtained from the dataset CSV file. The labels represent multiple lung diseases associated with each image, making it a multi-label classification problem. The correct mapping between images and labels is a necessary precondition for accurate evaluation.

Different input combinations are considered during testing, including images containing single diseases, multiple diseases, and normal cases. This helps in evaluating the robustness of the model across various scenarios.

In addition, the system requires pre-trained models (ResNet and EfficientNet) to be available and properly loaded before testing. The availability of necessary libraries such as TensorFlow, OpenCV, NumPy, and Pandas is also a precondition for smooth execution.

The testing environment must also meet hardware requirements, including sufficient memory and, preferably, GPU support to handle large-scale image processing efficiently.

By ensuring these input combinations and preconditions, the system is able to perform testing accurately and consistently across different scenarios.

6.4.5 Static vs. Dynamic Testing

Testing of the proposed system can be broadly categorized into static testing and dynamic testing, both of which play an important role in ensuring the correctness and reliability of the system.

Static testing is performed without executing the code and focuses on verifying the correctness of the system design, code structure, and documentation. In this project, static testing involves reviewing the implementation of preprocessing steps, model architecture, and threshold configurations. It also includes checking the correctness of data handling procedures, such as loading the dataset, mapping labels, and ensuring proper configuration of parameters. Static testing helps in identifying errors at an early stage, thereby reducing the chances of failures during execution.

Dynamic testing, on the other hand, involves executing the system and evaluating its behavior with actual input data. In this project, dynamic testing is carried out by providing chest X-ray images to the trained models and analyzing the predictions generated. The system processes each image, applies preprocessing, performs inference using ResNet and EfficientNet models, and generates outputs that are evaluated using performance metrics such as Accuracy, F1-score, and AUC.

Dynamic testing also includes validating the effectiveness of the fusion model, where predictions from both models are combined to improve overall performance. The system is tested under different input conditions to ensure consistent and reliable outputs.

Thus, both static and dynamic testing contribute to improving the quality of the system by identifying errors, validating functionality, and ensuring that the model performs as expected in real-world scenarios.

6.4.6 Types of Testing

Different types of testing are performed in this project to ensure that the proposed lung disease detection system functions correctly and efficiently. These testing methods help in verifying individual components as well as the overall system performance.

Unit Testing: Unit testing is performed to verify the correctness of individual components of the system. In this project, it includes testing functions such as image loading, preprocessing, threshold application, and prediction generation. Each module is tested independently to ensure it produces the expected output.

Integration Testing: Integration testing is carried out to ensure that different modules of the system work together properly. It involves combining components such as data preprocessing, model

inference, and evaluation metrics to verify that the complete pipeline functions without errors.

System Testing: System testing evaluates the entire system as a whole. In this project, it involves testing the complete workflow from input image processing to final disease prediction. This ensures that all components are integrated correctly and the system behaves as expected.

Performance Testing: Performance testing is conducted to analyze the efficiency and speed of the system. It evaluates how quickly the model processes input images and generates predictions. The use of GPU acceleration is also tested to ensure faster computation.

Validation Testing: Validation testing ensures that the system meets the desired performance criteria. It involves evaluating the model using metrics such as Accuracy, F1-score, and AUC to confirm that the predictions are reliable and accurate.

Regression Testing: Regression testing is performed to ensure that changes or improvements in the system do not negatively affect existing functionality. It verifies that previously tested components continue to work correctly after modifications.

By performing these different types of testing, the system is thoroughly evaluated for correctness, efficiency, and reliability, ensuring its suitability for real-world applications.

6.5 DESIGN OF TEST CASES AND SCENARIOS

The design of test cases and scenarios is an important step in the testing phase, as it ensures that all functionalities of the proposed system are thoroughly verified. In this project, test cases are designed to evaluate each stage of the lung disease detection system, from data loading to final prediction and performance evaluation.

The test cases are created based on different functional components of the system. The first scenario involves loading the dataset and verifying whether all chest X-ray images and corresponding labels are correctly read from the dataset files. This ensures that the system has access to valid input data for testing.

The next scenario focuses on model initialization, where the pre-trained ResNet and EfficientNet models are loaded into the system. This step verifies that the models are correctly configured and ready for inference.

Another important test case is image preprocessing, where input images are resized, normalized, and converted into the required format. This step ensures that all images are consistent and suitable for model input.

The prediction generation scenario evaluates the ability of the models to produce probability scores for multiple lung diseases. This is followed by threshold-based classification, where the predicted probabilities are converted into binary outputs using predefined threshold values.

The fusion model scenario is designed to test the ensemble approach, where predictions from both models are combined using weighted averaging. This helps in improving the overall prediction accuracy and robustness of the system.

Finally, the performance evaluation scenario measures the effectiveness of the system using metrics such as Accuracy, F1-score, and AUC. These metrics provide a detailed understanding of the model's performance across different disease classes.

Table 7: Types of tests

S.No	Test Case Name	Input	Expected Output
1	Dataset Loading	Dataset files	Successful data loading
2	Model Initialization	Model files	Models loaded correctly
3	Image Preprocessing	Raw images	Preprocessed images
4	Prediction Generation	Processed images	Probability scores
5	Threshold Classification	Predictions	Binary outputs
6	Fusion Model	Model outputs	Improved predictions
7	Performance Evaluation	Predictions & labels	Accuracy, F1-score, AUC

Table 8: Test Cases

Image ID	Type of Test	Expected Output	Actual Output	Result
img1 (03_0)	Classification	Hernia	Hernia	Pass
img2 (05_1)	Classification	No Finding	No Finding	Pass
img3 (08_0)	Classification	Cardiomegaly	Cardiomegaly	Pass
img4 (11_05)	Classification	Infiltration	Cardiomegaly, Effusion, Infiltration	Partial Pass

Table 7 The test results indicate that the proposed model performs accurately in most cases, correctly predicting diseases such as Hernia, No Finding, and Cardiomegaly. However, in some cases like *img4*, the model predicts additional diseases along with the expected one, resulting in a partial match. This occurs due to the multi-label nature of the dataset, where multiple conditions may coexist or share similar features. Overall, the system demonstrates strong performance with minor prediction overlaps.

6.3 VALIDATION TESTING

Validation and testing are performed to evaluate the performance and effectiveness of the proposed lung disease detection system. This phase ensures that the trained models are capable of accurately predicting multiple lung diseases from unseen chest X-ray images.

During the testing process, the input images are first preprocessed by resizing them to a fixed dimension, normalizing pixel values, and converting them into a suitable format for the models. These processed images are then passed through the ResNet and EfficientNet models to generate prediction probabilities for each of the 14 diseases.

Since the problem is a multi-label classification task, the predicted probabilities are converted into binary outputs using disease-specific threshold values. This threshold-based approach helps in improving classification accuracy by reducing false positives and false negatives. In addition to individual model predictions, a fusion technique is applied to combine the outputs of ResNet and EfficientNet. The fusion model uses weighted averaging to generate final predictions, thereby improving the overall robustness and performance of the system.

The performance of the models is evaluated using standard metrics such as Accuracy, F1-score, and Area Under Curve (AUC). Accuracy measures the overall correctness of predictions, while the F1-score provides a balance between precision and recall. The AUC metric evaluates the model's ability to distinguish between different disease classes and is calculated for each disease as well as averaged across all classes.

The testing results indicate that the ResNet model provides a strong baseline performance,

while the EfficientNet model enhances feature extraction and classification accuracy. The fusion model achieves the best performance by combining the strengths of both models, resulting in improved accuracy and consistency.

Furthermore, the validation results demonstrate that the system performs effectively across different disease categories. The mean AUC value confirms that the model has strong discriminative ability, and the F1-score indicates balanced performance across all classes.

Thus, the validation and testing phase confirms that the proposed system is reliable, accurate, and capable of assisting in the detection of lung diseases using chest X-ray images.

6.4 CONCLUSION

The testing and validation phase plays a crucial role in determining the effectiveness and reliability of the proposed lung disease detection system. In this project, the system has been thoroughly evaluated using the NIH Chest X-ray dataset to ensure accurate performance on unseen data.

The results obtained from testing demonstrate that the deep learning models, namely ResNet and EfficientNet, are capable of effectively extracting features and classifying multiple lung diseases. The implementation of threshold-based classification further improves the accuracy by reducing false predictions.

The fusion model, which combines the outputs of both ResNet and EfficientNet, shows improved performance compared to individual models. This ensemble approach enhances prediction accuracy, robustness, and consistency across different disease classes.

The evaluation metrics such as Accuracy, F1-score, and AUC confirm that the system performs reliably and maintains balanced performance across all classes. The validation results indicate that the proposed system is capable of generalizing well to unseen data.

Overall, the testing and validation phase confirms that the developed system is efficient, accurate, and suitable for practical applications in medical image analysis. It can serve as a supportive tool for early detection of lung diseases and assist healthcare professionals in diagnosis and decision-making.

7. CONCLUSION

The proposed Lung Disease Detection System has been accurately designed and implemented using advanced deep learning techniques to analyze chest X-ray images and identify multiple lung diseases. The system integrates a U-Net model trained on the JSRT dataset for accurate lung segmentation, which enhances the quality of input data by focusing only on the relevant lung regions. This is followed by the use of powerful classification models, ResNet and EfficientNet, whose predictions are combined using a fusion approach to improve overall performance and reliability.

The experimental results demonstrate that the system is capable of achieving high accuracy in multi-label classification, effectively detecting various lung diseases from the NIH Chest X-ray dataset. The use of preprocessing techniques, segmentation, and ensemble learning significantly contributes to reducing errors and improving model generalization.

Furthermore, the system has been thoroughly tested and validated using standard performance metrics such as Accuracy, F1-score, and AUC, ensuring its robustness and effectiveness. Although the current implementation is limited to offline evaluation, it provides a strong foundation for real-world medical applications.

In conclusion, the proposed system serves as an efficient and reliable tool for automated lung disease detection, with the potential to assist healthcare professionals in early diagnosis and decision-making. With further improvements and real-time integration, it can evolve into a practical solution for clinical use.

8. REFERENCES

1. M. Thapa and R. Kaur, “*An Explainable Deep Learning–Based Multi-Label Image Classification for Chest X-Rays*,” *Procedia Computer Science*, 2025.
Available: <https://doi.org/10.1016/j.procs.2025.04.505>
2. S. Sarker et al., “*Comprehensive Lung Disease Detection Using Deep Learning Models and Hybrid Chest X-Ray Data with Explainable AI*,” *arXiv*, 2024.
Available: <https://arxiv.org/abs/2505.16028>
3. S. Priyatharsini et al., “*Revolutionizing Lung Disease Diagnosis: A Hybrid Deep Learning Framework for Explainable Chest X-Ray Analysis*,” *SSRN*, 2024.
Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4754008
4. P. Rajpurkar et al., “*CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays*,” *arXiv:1711.05225*, 2017.
Available: <https://arxiv.org/abs/1711.05225>
5. T. T. Ifty et al., “*Explainable Lung Disease Classification from Chest X-Ray Images*,” *arXiv*, 2024.
Available: <https://arxiv.org/abs/2404.11428>
6. N. Veeramani et al., “*Explainable AI-Based Lung Disease Diagnosis Using Chest X-ray Images*,” *PMC*, 2025.
Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12475433/>
7. E. Mahamud et al., “*Explainable AI Model for COVID-19, Pneumonia and Tuberculosis Detection*,” *ScienceDirect*, 2024.
Available: <https://www.sciencedirect.com/science/article/pii/S2772662224001036>
8. Z. Naz et al., “*CNN-Based Explainable AI Framework for Pulmonary Disease Detection*,” *MDPI*, 2023.
Available: <https://www.mdpi.com/2072-6694/15/1/314>
9. “*Deep Learning for Pneumonia Detection in Chest X-ray Images: A Review*,” *PMC*, 2024.
Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11355845/>
10. Gao, L., Ma, X., Lin, J., & Callan, J. (2023). *Precise zero-shot dense retrieval without relevance labels*. *arXiv*. <https://arxiv.org/abs/2212.10496>